

بسم الله الرحمن الرحيم

Sudan University of Science and Technology



College of Computer Science and Information Technology

Digital E-invoicing and Tax Compliance Platform Prototype for the Sudan Taxation Chamber (STC)

Prepared by:

- 1. Ayman Salah Ismaiel haj-Elamin**
- 2. Samer Yousif Farah Balal**

Supervised By:

T. Ali Al-amin

June 2026

الايه

قال تعالى :

(وَقُلْ اَعْمَلُوا فَسَيَرَى اللّٰهُ

عَمَلَكُمْ وَرَسُولُهُ وَالْمُؤْمِنُونَ

وَسَتُرَدُّونَ اِلَى عَالِمِ الْغَيْبِ

وَالشَّهَادَةِ فَيُنَبِّئُكُمْ بِمَا كُنْتُمْ

تَعْمَلُونَ)

(105) سورة التوبه

الإهداء

إلى والدينا الكرام،

الذين كانوا بعد الله تعالى سببًا في وصولنا إلى هذه المرحلة، بما قدموه لنا من دعمٍ وتشجيعٍ وصبرٍ
وتضحيات.

إلى أساتذتنا الأفاضل،

الذين أناروا لنا طريق العلم والمعرفة، وقدموا لنا التوجيه والنصح خلال مسيرتنا الأكاديمية.

إلى مشرفنا الاستاذ علي الأمين،

على ما قدمه من إرشادٍ ومتابعةٍ وتوجيهٍ أسهم في إنجاز هذا المشروع.

إلى زملائنا وأصدقائنا،

الذين شاركونا رحلة الدراسة، وكانوا عونًا لنا في أوقات الجد والاجتهاد.

وإلى كل من ساندنا ولو بكلمة طيبة،

نهدي هذا العمل المتواضع، راجين من الله تعالى أن يكون علمًا نافعًا وخطوةً مباركة في طريق خدمة
وطننا ومجتمعنا.

Abstract

Manual invoicing and tax reporting can limit transparency, delay auditing, and increase the risk of tax evasion. This project addresses these challenges by presenting a prototype digital e-invoicing and tax compliance platform for the Sudan Taxation Chamber. The system includes a taxpayer client for generating standard invoices, applying XML canonicalization, creating invoice hashes, signing invoices, and submitting invoices to the backend. It also includes a backend that handles device enrolment, certificate issuance, invoice clearance, invoice reporting, validation, storage, and audit logging. The prototype demonstrates how digital invoicing, certificates, invoice chains, and secure API communication can improve tax compliance and provide a foundation for future digital tax transformation in Sudan.

المستخلص

يمكن أن تؤدي الفوترة اليدوية والتقارير الضريبية التقليدية إلى تقليل الشفافية، وتأخير عمليات المراجعة، وزيادة مخاطر التهرب الضريبي. يعالج هذا المشروع هذه التحديات من خلال تقديم نموذج أولي لمنصة رقمية للفوترة الإلكترونية والامتثال الضريبي لصالح ديوان الضرائب السوداني. يتضمن النظام تطبيقًا خاصًا بدافع الضرائب لإنشاء فواتير وفق معيار موحد، وتطبيق عملية التهيئة القياسية للملفات، وإنشاء بصمات الفواتير، وتوقيع الفواتير رقميًا، ثم إرسال الفواتير إلى الخادم الخلفي. كما يتضمن النظام خادمًا خلفيًا يتولى تسجيل الأجهزة، وإصدار الشهادات الرقمية، وتخليص الفواتير، والإبلاغ عنها، والتحقق من صحتها، وتخزينها، وتسجيل العمليات لأغراض المراجعة. يوضح النموذج الأولي كيف يمكن للفوترة الرقمية، والشهادات الرقمية، وسلاسل الفواتير، والاتصال الآمن عبر واجهات برمجة التطبيقات أن تسهم في تحسين الامتثال الضريبي وتوفير أساس للتحويل الرقمي المستقبلي في النظام الضريبي في السودان.

List of Abbreviations

API	Application Programming Interface
B2B	Business-to-Business
B2C	Business-to-Consumer
CA	Certificate Authority
CSR	Certificate Signing Request
CTC	Continuous Transaction Control
DER	Distinguished Encoding Rules
HTTPS	Hypertext Transfer Protocol Secure
ICV	Invoice Counter Value
JSON	JavaScript Object Notation
PEM	Privacy Enhanced Mail
PIH	Previous Invoice Hash
PKI	Public Key Infrastructure
REST	Representational State Transfer
SHA-256	Secure Hash Algorithm 256-bit
STC	Sudan Taxation Chamber
stc-cli	STC canonicalization command-line tool
TIN	Taxpayer Identification Number
TLS	Transport Layer Security
TLV	Tag-Length-Value
UBL	Universal Business Language
UUID	Universally Unique Identifier
VAT	Value Added Tax
XML	Extensible Markup Language
XML Digital Signature	XML signing standard
XAdES	XML Advanced Electronic Signatures
XSD	XML Schema Definition

Table of Contents

الإهداء.....	i
Abstract.....	iii
المستخلص	iv
1.1 Introduction.....	2
1.2 Definitions	2
1.3 Benefits.....	3
1.4 Problem Statement.....	3
1.5 Research Objectives.....	4
1.6 Scope of the Study	4
1.7 Significance of the Study	5
1.8 Research Methodology.....	5
1.9 Structure of the Research.....	7
Chapter Two Literature Review and Previous Studies	8
2.1 Introduction.....	9
2.2 Electronic Invoicing Systems	9
2.3 Continuous Transaction Control (CTC).....	10
2.4 Business-to-Business Document Standards	11
2.5 Structured E-Invoice Standards	17
2.6 Digital Signature and Security Technologies.....	18
2.7 REST APIs and Validation Server Architectures	19
2.8 Containerization and Scalable Deployment.....	20
2.9 Sudan Infrastructure Challenges.....	20
2.10 Previous Studies and Existing Systems	21
Chapter Three System Analysis and Design.....	24
3.1 Introduction.....	25
3.2 System Overview	25
3.3 Existing System Analysis	26
3.4 Proposed System Analysis	27
3.5 Stakeholders and Actors.....	28
3.6 Functional Requirements	28
3.7 Non-Functional Requirements.....	30

3.8 System Architecture.....	30
3.9 System Modules.....	32
3.10 Use Case Model	39
3.11 Database Design	39
3.12 API Design	40
3.13 Security Design.....	42
3.14 Adaptation to Sudanese Infrastructure	42
3.15 Deployment Design	43
3.16 Prototype Limitations	43
Chapter Four System Implementation	44
4.1 Introduction.....	45
4.2 Development Tools and Technologies	45
4.3 Implementation Setup.....	46
4.4 Onboarding Implementation.....	47
4.5 Enrollment Implementation.....	49
4.6 Invoice Generation and Signing Implementation	51
4.7 Clearance and Reporting Implementation	53
4.8 Server-Side Validation and Storage Implementation.....	55
4.9 Sandbox mode	56
4.10 Auditing and Logs Implementation.....	57
4.11 Deployment and Performance	58
Chapter Five Results, Recommendations and Conclusion.....	60
5.1 Results	61
5.2 Recommendations	64
5.3 Conclusion.....	64
References.....	66

List of Tables

Table 2.1: Standard Documents in EDI Standards	12
Table 2.2: Standard Documents in XML-Based Business Standards	13
Table 2.3: Documents in E-Invoicing Frameworks	14
Table 2.4: Industry-Specific Standard Documents.....	15
Table 2.5: Comparison of B2B Document Standard Categories	16
Table 2.6: Literature Review of E-Invoicing Systems	22
Table 2.7: Summary of Related Technologies	23
Table 3.1: Main System Actors and Responsibilities.....	28
Table 3.2: Condensed Functional Requirements.	29
Table 3.3: Non-Functional Requirements.....	30
Table 3.4: Main Use Cases.	39
Table 3.5: Main Data Entities.	40
Table 3.6: Main API Endpoints.	41
Table 4.1: Implemented backend invoice validation steps.	56
Table 5.1 server performance test output.....	62
Table 5.2: Estimated invoice payload storage requirements.	63

Table of Figures

Fig 3.1: High-Level STC E-Invoicing System Overview.	25
Fig 3.2: Proposed Digital E-Invoicing Workflow.....	26
Fig 3.3: Taxpayer Client Layered Architecture.	31
Fig 3.4: Device Enrollment and Certificate Issuance Sequence.	33
Fig 3.5: Invoice Preparation Pipeline.....	34
Fig 3.6: Clearance Workflow.	36
Fig 3.7: Reporting Workflow.	37
Fig 3.8: Backend Invoice Validation Pipeline.	38
Fig 3.9: Logical Entity Relationship Diagram.	40
Fig 3.10: Deployment Diagram.	43
Fig 4.1: Taxpayer client startup screen.	47
Fig 4.2: Taxpayer login screen.	48
Fig 4.3: One-time enrollment token generation.	48
Fig 4.4: Device enrollment form.....	50
Fig 4.5: Certificate enrollment success screen.	51
Fig 4.6: Invoice data entry screen.	52
Fig 4.7: Signed invoice generation result.	53
Fig 4.8: Clearance workflow screen.....	54
Fig 4.9: Reporting workflow screen.....	55
Fig 4.10: Stored backend invoice records.	56
Fig 4.11: Sandbox response screen.....	57
Fig 4.12: Runtime logging output.	58
Fig 4.13: Backend deployment.	59
Fig 5.1: Server performance test output.	62

Chapter One

Introduction

1.1 Introduction

Sudan's taxation environment requires a practical digital modernization approach rather than a generic technology solution. The continuing disruption of formal economic activity, communication services, and business operations has made reliable transaction records more important for both taxpayers and the Sudan Taxation Chamber (STC). Reuters reported that Sudan's GDP contracted by 29% in 2023 and a further 13.5% in 2024, while the national currency lost more than 80% of its value during the conflict [1]. These conditions increase the need for verifiable commercial records, transparent Value Added Tax (VAT) reporting, and audit trails that can survive fragmented manual processes.

Electronic invoicing is relevant to this context because it turns invoices into structured, machine-processable records that can be validated automatically and exchanged through secure digital channels. This study therefore focuses on a Sudan-oriented prototype for the STC, where registered taxpayers can generate compliant invoices, enroll their devices securely, submit invoices to a central backend, and receive validation outcomes through clearance and reporting workflows.

1.2 Definitions

1.2.1 Electronic Invoicing

Directive 2014/55/EU defines an electronic invoice as an invoice issued, transmitted, and received in a structured electronic format that allows automatic and electronic processing [2]. This definition distinguishes true electronic invoices from scanned paper invoices or ordinary PDF files. In this research, electronic invoicing is implemented through structured UBL 2.1 invoice data, where OASIS defines the Invoice document as a standard document used to request payment and publishes the corresponding UBL-Invoice-2.1 schema [11].

1.2.2 Tax Compliance Platform

For this study, a tax compliance platform is defined operationally as a digital system that supports taxpayer onboarding, invoice generation, submission, rule-based validation, certificate verification, audit logging, and compliance-oriented record management. The proposed STC platform consists of a taxpayer client application, a centralized STC backend server, secure device enrollment, certificate-based trust, and database-backed invoice records.

1.3 Benefits

A Sudan-oriented e-invoicing platform can provide three main categories of benefits. For the tax authority, it supports better visibility over submitted invoices, faster validation of invoice structure, stronger VAT traceability, and centralized audit records. For businesses, it reduces paper-based effort, organizes invoice submission status, and provides clearer compliance steps. Technically, it enables HTTPS-based communication, structured UBL invoice exchange, certificate-based onboarding, automated validation, and future integration with broader tax administration services.

1.4 Problem Statement

The problem addressed by this research is not the entire Sudanese taxation system, but a focused technical-administrative gap: the absence of a Sudan-oriented prototype that combines structured invoice generation, secure taxpayer or device onboarding, centralized invoice validation, and auditable invoice storage in one workflow. Existing manual or disconnected processes make invoice verification slow, weaken VAT traceability, and increase the burden of auditing.

This gap is significant because Sudan's commercial environment has been affected by economic contraction, infrastructure damage, currency deterioration, and unstable digital services. When invoices are paper-based or fragmented across unconnected tools, the STC has limited ability to verify invoice authenticity, monitor VAT-related transactions, detect inconsistencies,

or maintain reliable audit trails. Therefore, the research problem is: How can a secure prototype e-invoicing platform be designed and evaluated for the Sudanese context to support taxpayer onboarding, structured invoice submission, automated validation, and centralized auditability?

1.5 Research Objectives

The general objective of this research is to design and evaluate a prototype digital e-invoicing and tax compliance platform suitable for the Sudan Taxation Chamber context.

1. To identify the functional, interoperability, and security requirements of a Sudan-oriented e-invoicing platform.
2. To design a prototype artifact that connects a taxpayer-side compliance client with a centralized STC backend server.
3. To examine the suitability of UBL 2.1, PKI-based enrollment, REST/HTTPS communication, and XAdES-related signing concepts for the proposed workflow.
4. To evaluate the prototype in a controlled sandbox environment using functional, security, performance, reliability, and usability indicators.
5. To derive design lessons that can guide future development of more robust digital tax compliance systems in Sudan.

1.6 Scope of the Study

1.6.1 Business Scope

The business scope is limited to the invoice-compliance lifecycle between registered taxpayers and the STC prototype environment. It includes invoice generation, device enrollment, invoice submission, clearance, reporting, validation feedback, and audit logging. It excludes payment settlement, banking integration, customs integration, national identity integration, large-scale ERP integration, and nationwide rollout.

1.6.2 Legal Scope

The system is a compliance-oriented academic prototype and does not claim full legal enforceability. It demonstrates technical mechanisms related to authenticity, integrity, traceability, and audit support. A production national system would require formal legislation, official governance rules, data retention policies, privacy controls, certification procedures, and approval by the relevant Sudanese authorities.

1.6.3 Deployment Assumptions

The prototype assumes a controlled sandbox or pilot-like environment, a limited number of test users, stable enough connectivity for evaluation, test certificates or simulated trust material, and a centrally managed backend server. The design emphasizes feasibility and proof of concept rather than full production resilience, disaster recovery, or national-scale availability.

1.7 Significance of the Study

This research is significant because it adapts international e-invoicing concepts to a Sudanese institutional problem. Governmentally, it demonstrates how centralized invoice validation can improve auditability and transaction visibility. For businesses, it shows a clearer digital path for invoice preparation and submission. Technically, it integrates secure backend engineering, REST APIs, PKI enrollment, structured invoice standards, and cryptographic validation concepts. Academically, it presents a Sudan-oriented artifact that can be studied, evaluated, and improved in later work.

1.8 Research Methodology

The study follows a Design Science Research orientation because its main contribution is an artifact: a prototype e-invoicing and tax compliance platform. Design Science Research is appropriate for information systems projects that create and evaluate purposeful artifacts to address relevant practical problems [3], [4].

1.8.1 Data Collection

Data collection is document-based and requirement-oriented. It uses standards and technical references on e-invoicing, UBL 2.1, PKI, digital signatures, REST APIs, and related tax-compliance models. It also uses Sudan-specific contextual evidence to identify constraints such as manual processes, fragmented records, and infrastructure limitations. These inputs are converted into functional and security requirements for the prototype.

1.8.2 Evaluation Methods and metrics

The prototype is evaluated using scenario-based testing. Each scenario represents a major workflow in the proposed electronic invoicing platform, such as taxpayer enrollment, invoice generation, invoice submission, clearance processing, reporting processing, invalid invoice rejection, and audit record storage.

The evaluation uses the following metrics:

1. Functional accuracy: whether the system correctly completes the intended workflows, including enrollment, invoice submission, clearance, reporting, and database storage.
2. Validation effectiveness: whether the system accepts structurally valid invoices and rejects invalid, incomplete, or non-compliant submissions.
3. Security effectiveness: whether certificate-based enrollment, invoice hash verification, and signature-related checks behave as expected during testing.
4. Response time: the time required by the backend to process key operations such as enrollment, clearance, and reporting requests.
5. Reliability: whether repeated test scenarios complete successfully without unexpected system failure.
6. Usability: whether users can complete the main taxpayer-side actions, such as enrollment, invoice preparation, and submission, with reasonable clarity and effort.

These metrics are used to determine whether the prototype satisfies the research objectives and whether the proposed design is suitable as a foundation for future Sudan-oriented electronic invoicing development.

1.8.3 Experimental Setup

The prototype is evaluated in a controlled sandbox environment that simulates interaction between a taxpayer client and the Sudan Taxation Chamber server. The setup includes a Flutter taxpayer client, a Rust/Actix Web backend, a PostgreSQL database, HTTPS/REST communication, UBL 2.1 invoice structures, PKI-based enrollment, and XAdES-related signing concepts.

The client generates and submits invoices, while the backend validates invoice data, processes clearance or reporting requests, and stores audit-related records. The setup is limited to prototype testing and does not represent nationwide deployment, banking integration, or official legal enforcement.

1.9 Structure of the Research

This thesis is organized into five chapters:

- Chapter 1 introduces the study background, problem statement, objectives, scope, significance, and methodology.
- Chapter 2 reviews previous studies, e-invoicing systems, tax compliance infrastructures, invoice standards, and related technologies.
- Chapter 3 presents the methodology, system analysis, architecture, database design, API workflows, and security model.
- Chapter 4 explains the implementation of the taxpayer client, STC backend, enrollment, validation, clearance, reporting, and database integration.
- Chapter 5 presents the results, evaluation, limitations, recommendations, and future work.

Chapter Two

Literature Review and Previous Studies

2.1 Introduction

Electronic invoicing and digital tax compliance systems have become important components of modern tax administration. Governments increasingly use digital platforms to improve tax transparency, reduce fraud, automate Value Added Tax (VAT) monitoring, and provide better visibility over business transactions. Traditional paper-based taxation is usually slower, more error-prone, and harder to audit because invoices can be lost, manipulated, delayed, or omitted.

Modern e-invoicing systems combine structured business document standards, cryptographic validation, centralized compliance servers, REST APIs, secure communication, and audit storage. Standards such as Universal Business Language (UBL), XML Schema Definition (XSD), Public Key Infrastructure (PKI), XML Digital Signature, and XAdES are widely used to support interoperability, integrity, and authenticity of invoice data [11], [17], [18].

This chapter reviews the main concepts, document standards, technologies, and previous studies related to electronic invoicing and digital tax administration. Special focus is given to Business-to-Business (B2B) document standards because they define the formal document types, such as purchase orders, invoices, shipping notices, and acknowledgments, that allow business systems and tax platforms to exchange data consistently.

2.2 Electronic Invoicing Systems

Electronic invoicing refers to the digital creation, exchange, validation, and storage of invoices using structured electronic formats instead of paper documents or unstructured PDF files. A true electronic invoice is machine-readable and can be automatically checked by accounting systems, enterprise systems, or tax authority platforms.

The main objectives of e-invoicing systems are to improve tax transparency, reduce tax evasion, automate VAT reporting, minimize manual errors, improve auditability, and enable near real-time transaction monitoring. OECD digital tax administration literature describes this direction as moving toward compliance-by-design, where tax requirements are integrated directly into business systems instead of being handled only after transactions are completed [7].

An e-invoicing workflow usually includes invoice generation, document structuring, digital signing or hashing, secure submission, validation by the receiving system, response handling, and long-term storage. In tax authority systems, this workflow is usually connected to clearance or reporting models.

2.3 Continuous Transaction Control (CTC)

Continuous Transaction Control (CTC) refers to tax models where invoice data is monitored, validated, or reported to the tax authority during or shortly after the transaction. CTC systems are increasingly used because they give tax authorities earlier visibility over taxable transactions and reduce the delay between invoice issuance and tax reporting.

2.3.1 Clearance Model

In the clearance model, an invoice is submitted to the tax authority or a certified clearance platform before it becomes legally valid or before it is finally delivered to the buyer. The authority validates the invoice structure, tax fields, digital signature, and compliance rules, then returns an approved or cleared invoice. This model provides strong control because invalid invoices can be rejected early.

2.3.2 Reporting Model

In the reporting model, the seller may issue the invoice first and then report invoice data to the tax authority within a defined time or workflow. Reporting is easier to deploy and can reduce dependency on continuous connectivity, but it provides less immediate control compared to full clearance.

2.3.3 Hybrid Model

Hybrid systems combine both models. For example, B2B invoices may require clearance while B2C or simplified invoices may only require reporting. This approach is useful for gradual deployment because it balances control, infrastructure limits, and taxpayer readiness.

2.4 Business-to-Business Document Standards

Business-to-Business document standards are formal rules used to represent and exchange recurring business documents between organizations. They define document syntax, data fields, message types, identifiers, and exchange expectations. Common B2B documents include purchase orders, invoices, credit notes, despatch or shipping notices, receiving advice, payment messages, and acknowledgments.

B2B standards are important for e-invoicing because the invoice is not an isolated document. It is part of a wider order-to-cash process that may start with a purchase order and continue through shipment, receipt, invoicing, payment, and confirmation. For tax compliance platforms, standardized documents make automated validation, interoperability, and auditability possible.

2.4.1 EDI Standards

Electronic Data Interchange (EDI) is one of the oldest forms of structured B2B exchange. EDI standards such as ANSI X12 and UN/EDIFACT define transaction sets or message types for commercial processes. ANSI X12 uses numeric transaction codes, while EDIFACT uses named message identifiers. UNECE describes UN/EDIFACT as an international set of standards, directories, and rules for electronic data interchange [9], while X12 defines transaction sets for business functions such as ordering, invoicing, shipping, and acknowledgments [10].

Table 2.1: Standard Documents in EDI Standards

Business Document	ANSI X12 Example	EDIFACT Example	Purpose
Purchase Order	EDI 850	ORDERS	Buyer sends an order to a supplier
Invoice	EDI 810	INVOIC	Seller requests payment for goods or services
Advance Ship Notice / Despatch Advice	EDI 856	DESADV	Supplier communicates shipment details
Purchase Order Acknowledgment	EDI 855	ORDRSP	Supplier accepts, rejects, or changes an order
Receiving Advice	EDI 861	RECADV	Buyer confirms received goods
Functional / Control Acknowledgment	EDI 997	CONTRL	Receiver confirms receipt or reports message status

For the STC prototype, EDI is mainly a comparative background. It shows that invoice exchange belongs to a larger business document lifecycle. However, XML-based standards such as UBL are more suitable for this project because the prototype focuses on XML invoices, REST APIs, validation, digital signatures, and tax compliance workflows.

2.4.2 XML-Based Business Standards

XML-based business standards represent business documents using XML schemas. They are more readable than classic EDI and can be validated using formal schema definitions. Universal Business Language (UBL) is the most relevant XML business standard for this project. OASIS UBL 2.1 defines common business documents such as Order, Invoice, Credit Note, Despatch Advice, Receipt Advice, and Application Response [11].

Table 2.2: Standard Documents in XML-Based Business Standards

Standard	Standard Documents	Purpose
UBL 2.1	Order, Invoice, Credit Note, Debit Note, Despatch Advice, Receipt Advice, Application Response	General structured business document exchange
cXML	PunchOut Setup Request, Order Request, Invoice Detail Request	Procurement and supplier network integration
OAGIS	Purchase Order, Shipment, Invoice, Payment	Enterprise application integration
GS1 XML	Order, Order Response, Invoice, Despatch Advice	Retail and supply-chain document exchange

In the STC prototype, UBL 2.1 is used as the main invoice document structure because it supports machine-readable invoice data, schema validation, and interoperability with tax compliance workflows.

2.4.3 E-Invoicing Framework Standards

E-invoicing framework standards go beyond document syntax. They define invoice semantics, validation rules, code lists, response messages, exchange networks, and compliance workflows. These frameworks ensure that invoices contain the required business, tax, and identity information needed for automated processing and legal compliance. Peppol BIS Billing 3.0, for example, supports standardized electronic invoice and credit note exchange based on the European EN 16931 semantic model [12], [13]. Similarly, ZATCA combines XML structure, security requirements, QR information, validation rules, and invoice categories such as standard tax invoices and simplified tax invoices [8]. Therefore, e-invoicing frameworks provide a complete compliance environment that connects invoice generation, validation, transmission, authority response, and auditability.

Table 2.3: Documents in E-Invoicing Frameworks

Framework	Documents / Messages	Purpose
Peppol BIS Billing 3.0	Invoice, Credit Note, Response Message	Interoperable invoice exchange
EN 16931	Core invoice semantic model and business rules	European invoice interoperability
ZATCA-style framework	Standard Tax Invoice, Simplified Tax Invoice, Clearance Response, Reporting Acknowledgment, QR data	Tax authority compliance
Factur-X / ZUGFeRD	Hybrid PDF/XML Invoice	Human-readable and machine-readable invoicing

This category is the closest conceptual match to the STC platform because the prototype validates structured invoices, verifies digital signatures, supports clearance and reporting, and returns compliance responses.

2.4.4 Financial Messaging Standards

Financial messaging standards are not invoice standards, but they support future payment integration and reconciliation. ISO 20022 defines structured messages for payments, cash management, securities, and other financial services [14].

2.4.5 Industry-Specific Standards

Some industries require specialized B2B standards because general invoice structures may not fully represent sector-specific processes. GS1 XML, for example, supports supply-chain messages such as order, invoice, order response, and dispatch advice [15]. Similar domain-specific standards exist in healthcare, aviation, manufacturing, and insurance.

Table 2.4: Industry-Specific Standard Documents

Industry	Standard	Example Documents	Purpose
Retail / Supply Chain	GS1 XML / GS1 EDI	Order, Invoice, Despatch Advice, Receiving Advice	Product and supply-chain exchange
Healthcare	HL7 / FHIR	Claim, Observation, Encounter, Patient Record	Healthcare data exchange
Aviation Cargo	IATA Cargo-XML	Air Waybill, Shipment Status, Cargo Manifest	Air cargo processing
Insurance	ACORD	Policy, Claim, Billing Notice	Insurance document exchange

For STC, these standards are not immediate design targets. They are useful as future extension references if a national e-invoicing platform later requires sector-specific invoice profiles.

2.4.6 Modern Web-Friendly Formats

Modern web-friendly formats support API-based exchange between applications. JSON is widely used for REST API request and response bodies, while YAML and JSON are commonly used to describe APIs through OpenAPI. The OpenAPI Specification defines a standard way to describe HTTP APIs, including endpoints, request bodies, responses, and security schemes [16].

In the STC prototype, JSON is used for API communication, while UBL XML represents the actual invoice document. This separates the transport layer from the legally structured invoice layer.

2.4.7 Comparative Analysis of B2B Document Standards

Table 2.5: Comparison of B2B Document Standard Categories

Category	Examples / Documents	Strengths	STC Relevance
EDI Standards	ANSI X12, EDIFACT; EDI 850, 810, 856, 997;	Mature and widely used in enterprise exchange	Background comparison
XML-Based Standards	UBL, cXML, OAGIS; Invoice, Order, Credit Note, Despatch Advice	Structured, schema-validatable, extensible	Core invoice document layer
E-Invoicing Frameworks	Peppol, EN 16931, ZATCA; Tax Invoice, Credit Note, Clearance Response	Compliance-oriented and rule-based	Core compliance model
Financial Messaging	ISO 20022, SWIFT; pain.001, pacs.008, camt.053, MT103	Strong payment and reconciliation semantics	Future integration
Industry-Specific Standards	GS1, HL7, IATA Cargo-XML; Order, Claim, Air Waybill	Domain-specific accuracy	Future sector extension
Web-Friendly Formats	JSON, YAML, OpenAPI; API requests, responses, error messages	Lightweight and developer-friendly	API communication layer

2.5 Structured E-Invoice Standards

Structured invoice standards define how invoice information should be represented so that it can be processed automatically. In e-invoicing systems, structure is essential because validation servers must check mandatory fields, totals, tax values, supplier identity, customer identity, and invoice references consistently.

2.5.1 Universal Business Language (UBL 2.1)

UBL 2.1 is an XML-based standard for electronic business documents such as invoices, orders, credit notes, and despatch advice. A UBL invoice organizes invoice data into XML components such as supplier information, customer information, invoice lines, tax totals, legal monetary totals, and document references [11].

- <Invoice>
- <AccountingSupplierParty>
- <AccountingCustomerParty>
- <TaxTotal>
- <InvoiceLine>
- <LegalMonetaryTotal>

The advantages of UBL include interoperability, machine readability, extensibility, and compatibility with XML schema validation. For this reason, UBL 2.1 is suitable for the proposed STC prototype.

2.5.2 XML Schema Definition (XSD)

XML Schema Definition is used to define valid XML structure, data types, element order, and mandatory fields. In e-invoicing, XSD validation ensures that invoices follow the approved document structure before further business or security validation is performed.

2.5.3 JSON Invoice Payloads

JSON is commonly used for API communication because it is lightweight and easy to process in modern web and mobile applications. However, JSON alone does not provide the same standardized legal invoice semantics as UBL unless it is controlled by a specific schema or framework. Therefore, this project uses JSON for API payloads and UBL XML for the invoice document.

2.5.4 UUID and QR Code Systems

Modern e-invoicing systems often use UUIDs, invoice counters, previous invoice hashes, and QR codes. UUIDs reduce duplicate submission risks, invoice counters support sequencing, previous invoice hashes support tamper-evidence, and QR codes allow invoice data to be verified by customers or inspectors.

2.6 Digital Signature and Security Technologies

Security is central to e-invoicing because invoice data may affect tax liability, financial records, and legal accountability. A secure e-invoicing system must protect authenticity, integrity, non-repudiation, and secure communication.

2.6.1 Public Key Infrastructure (PKI)

Public Key Infrastructure provides a trust framework using certificate authorities, public/private key pairs, digital certificates, certificate signing requests, and trust chains. In an e-invoicing platform, PKI binds an invoice or taxpayer device to a verified identity, ensuring that submitted invoices come from an authorized source. It also allows the system to verify digital signatures, confirm invoice integrity, and detect unauthorized modification after signing.

2.6.2 Cryptographic Hashing

Cryptographic hashing creates a fixed-size fingerprint of invoice data. If the invoice content changes, the hash changes. SHA-256 is widely used for integrity verification and invoice chaining in modern digital systems.

2.6.3 XML Digital Signature and XAdES

XML Digital Signature defines an XML syntax for signing XML data, including SignedInfo, Reference, DigestValue, SignatureValue, and KeyInfo elements [18]. XAdES extends XML signatures with additional properties useful for advanced electronic signatures, such as signing certificate information and qualifying properties [17]. These standards are relevant because UBL invoices are XML documents.

2.6.4 XML Canonicalization

XML canonicalization transforms XML into a deterministic byte representation before hashing or signing. This is required because the same XML document may be serialized differently due to whitespace, namespace ordering, or attribute formatting. Canonicalization ensures that the invoice hash and signature are based on a stable representation.

2.6.5 HTTPS and Token Authentication

HTTPS protects data in transit using TLS encryption. Authentication tokens and API authorization mechanisms help ensure that only authorized taxpayers or enrolled devices can submit invoices to the tax authority platform.

2.7 REST APIs and Validation Server Architectures

Modern e-invoicing infrastructures commonly use REST APIs to connect taxpayer systems, client applications, and tax authority servers. REST APIs provide stateless communication, simple integration, and compatibility with web and mobile applications.

A centralized validation server normally performs schema validation, business rule validation, duplicate detection, hash verification, digital signature verification, certificate validation, clearance or reporting processing, database persistence, and audit logging.

In the STC prototype, the backend server represents the tax authority side. It receives signed invoices, validates UBL structure, checks invoice hashes and signatures, verifies certificate binding, maintains invoice state, and supports clearance and reporting workflows.

2.8 Containerization and Scalable Deployment

Containerization improves deployment portability by packaging an application with its dependencies. Docker containers help isolate services, simplify deployment, and make the runtime environment more consistent across development and production environments [19]. Large-scale container orchestration concepts, such as those discussed in Kubernetes-related research, also support scalable and manageable cloud deployment [20].

For a digital tax compliance platform, containerization is useful because backend services, databases, validation engines, and supporting tools can be deployed in a predictable environment. This improves maintainability and future scalability.

2.9 Sudan Infrastructure Challenges

Implementing e-invoicing in Sudan requires consideration of local infrastructure constraints. Challenges may include unstable internet connectivity, inconsistent electrical power, limited enterprise digitization, limited cloud maturity, and low digital adoption among small and medium-sized businesses. These constraints can affect invoice generation, submission, and verification, especially outside major urban areas. Therefore, a Sudan-oriented platform should support gradual adoption, simplified onboarding, lightweight API communication, and sandbox testing before full deployment.

2.10 Previous Studies and Existing Systems

2.10.1 Saudi Arabia ZATCA E-Invoicing Framework

Saudi Arabia implemented a national e-invoicing framework through the Zakat, Tax and Customs Authority. The framework uses structured invoices, QR data, security features, and phased implementation. It is important for this research because it demonstrates a regional tax authority model based on invoice digitization, validation, and compliance workflows [8].

2.10.2 Kenya eTIMS VAT Reporting

Kenya Revenue Authority introduced the Electronic Tax Invoice Management System to improve VAT reporting and electronic tax invoice management. It is relevant because it shows how an African tax authority can use electronic invoice submission to improve transparency and compliance [21].

2.10.3 Nigeria TaxPro-Max

Nigeria's Federal Inland Revenue Service introduced TaxPro-Max as part of digital tax administration. It supports online tax services and digital interaction between taxpayers and the tax authority. It is relevant as an African example of tax administration modernization [22].

2.10.4 European Union EN 16931 and Peppol

The European Union EN 16931 standard defines a semantic model for electronic invoicing, while Peppol provides interoperability specifications for exchanging invoices and related documents. This is relevant because it shows how invoice semantics, syntax, and exchange infrastructure can be standardized across organizations and countries [12], [13].

2.10.5 OECD Digital Compliance Framework

OECD studies on tax administration emphasize digital transformation, data-driven compliance, and integration of tax processes into business systems. This supports the conceptual direction of the proposed STC platform as a prototype for improved transparency and compliance-by-design [7].

Table 2.6: Literature Review of E-Invoicing Systems

Study / System	Description	Technology / Framework	Relevance to STC
Saudi Arabia ZATCA	National e-invoicing framework with phased implementation	XML, QR codes, security features, APIs	Regional reference for clearance/reporting concepts
Kenya eTIMS	Electronic tax invoice management and VAT reporting	Tax invoice submission and digital reporting	African reference for tax digitization
Nigeria TaxPro-Max	Digital tax administration platform	Online taxpayer services and digital filing	African reference for administration modernization
EU EN 16931 / Peppol	Semantic invoice model and interoperability framework	UBL/CII, business rules, Peppol exchange	Strong standards reference
OECD Tax Administration 3.0	Digital transformation and compliance-by-design framework	Digital platforms and integrated compliance	Conceptual foundation for digital tax systems

Table 2.7: Summary of Related Technologies

Technology	Function in E-Invoicing	Use in STC Prototype
UBL 2.1	Structured XML invoice representation	Core invoice document format
XSD	Structural XML validation	Invoice schema validation
PKI	Identity and trust infrastructure	Device/taxpayer certificate enrollment
XML Signature / XAdES	Invoice authenticity and integrity	Signed invoice validation
REST APIs / JSON	System communication	Enrollment, clearance, and reporting requests
Docker	Portable deployment	Backend deployment and future scalability

Chapter Three

System Analysis and Design

3.1 Introduction

This chapter presents the system analysis and design of the Digital E-Invoicing and Tax Compliance Platform Prototype for the Sudan Taxation Chamber (STC). It explains the system context, actors, requirements, architecture, workflows, database model, API design, security design, deployment model, and prototype limitations.

The proposed system is designed as a secure electronic tax compliance ecosystem rather than a simple invoicing application. The taxpayer-side client prepares compliant invoices by generating UBL 2.1 XML, canonicalizing the XML, calculating hashes, applying XAdES-style digital signatures, generating QR data, and submitting invoices to the STC backend. The authority-side server validates submitted invoices, checks certificate binding, verifies integrity, maintains invoice chain state, and stores accepted records for audit purposes.

3.2 System Overview

The proposed platform follows a client-server model. The taxpayer client application runs on the taxpayer device and performs invoice preparation. The STC compliance server runs as the authority-side backend and performs enrollment, validation, clearance, reporting, and persistence. The backend database stores taxpayer, device, invoice, and audit-related records, while the client keeps local SQLite records for invoice history, Invoice Counter Value (ICV), and Previous Invoice Hash (PIH).

High-Level STC E-Invoicing System Overview

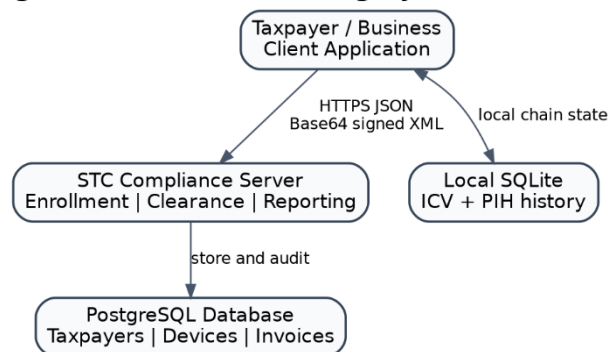


Fig 3.1: High-Level STC E-Invoicing System Overview.

The system supports two invoice processing modes. Clearance mode is used when an invoice must be validated by STC before final acceptance. Reporting mode is used when an invoice is reported to STC for recording after issuance. Both modes share the same core validation concepts: structured XML validation, canonicalization, hash verification, signature verification, certificate binding, and invoice chain checking.

Proposed Digital E-Invoicing Workflow

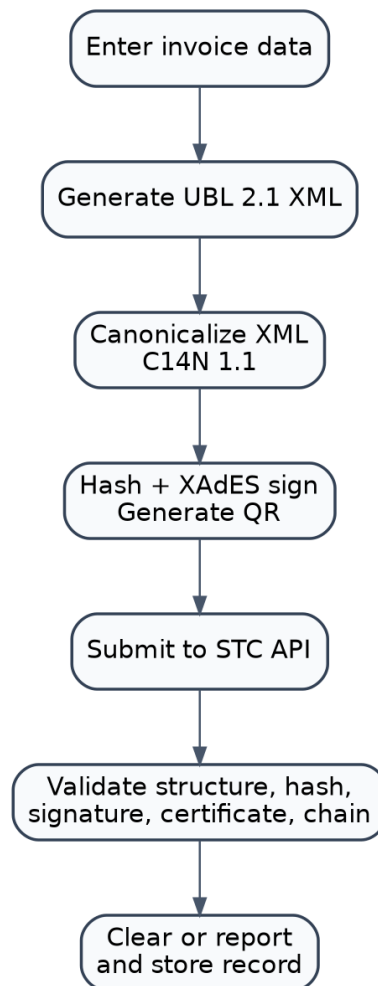


Fig 3.2: Proposed Digital E-Invoicing Workflow.

3.3 Existing System Analysis

The existing invoicing and tax filing workflow in Sudan is still largely dependent on manual or semi-manual processes. Businesses may issue paper invoices, informal receipts, or periodic declarations. This limits STC visibility

over taxable transactions and makes auditing slow, incomplete, and highly dependent on manual inspection.

The main weaknesses of the existing system are incomplete invoice records, weak transaction traceability, delayed tax visibility, possible underreporting of taxable sales, manual filing errors, and limited ability to detect missing or manipulated invoices. Because records are not consistently submitted in structured digital form, the tax authority cannot reliably validate invoices in real time or maintain a complete transaction trail.

3.4 Proposed System Analysis

The proposed system replaces informal invoice handling with a structured digital compliance workflow. Each taxpayer device must be enrolled before it can issue signed invoices. Each invoice is generated as UBL 2.1 XML, linked to the previous invoice through PIH, assigned an ICV, signed using the taxpayer private key, and submitted to STC through a secure API.

The design principle is that compliance should be enforced by the system workflow, not only by manual trust. An accepted invoice must have a valid structure, a verifiable hash, a valid XAdES-style signature, a certificate associated with an enrolled taxpayer device, and a continuous invoice chain. This improves integrity, traceability, and auditability while keeping the prototype realistic for Sudanese infrastructure constraints.

This workflow also reduces the chance of missing or manipulated invoices because every invoice is connected to a registered device and to the previous invoice record. By combining UBL validation, digital signatures, certificate binding, and invoice chaining, the system turns tax compliance into a technical verification process instead of a purely manual review.

3.5 Stakeholders and Actors

Table 3.1: Main System Actors and Responsibilities.

Actor	Description	Main Responsibilities
Taxpayer / Business User	The business entity issuing invoices.	Enter invoice data, generate invoices, enroll device, submit invoices, review results.
Taxpayer Device	The physical or logical device used by the taxpayer client.	Generate private key and CSR, store certificate, sign invoices, maintain ICV and PIH.
STC Compliance Server	Authority-side backend that validates and records invoices.	Enroll devices, issue certificates, validate invoices, clear/report invoices, store records.
STC Officer	Tax authority user who monitors compliance records.	Review submissions, investigate invalid invoices, access audit information in a future dashboard.
Compliance Administrator	Technical/admin role managing rules and policies.	Maintain validation rules, certificate policies, sandbox behavior, and security settings.
Database System	Persistence layer for authority and client records.	Store taxpayers, devices, challenges, invoices, local invoice history, and audit data.

3.6 Functional Requirements

Functional requirements define the services that the system must provide. For clarity, the requirements are grouped according to the taxpayer client, enrollment process, and STC server.

Table 3.2: Condensed Functional Requirements.

ID	Subsystem	Requirement
FR-01	Client	Allow the taxpayer to enter supplier, customer, and invoice item information.
FR-02	Client	Generate UBL 2.1 XML invoices with invoice ID, UUID, ICV, PIH, tax totals, and invoice lines.
FR-03	Client	Canonicalize invoice XML, calculate SHA-256 invoice hash, generate XAdES-style signature.
FR-04	Client	Submit signed invoices to clearance or reporting endpoints using HTTPS JSON requests.
FR-05	Client	Store local invoice history and chain state using SQLite.
FR-06	Enrollment	Generate the private key locally and submit only the CSR and enrollment token to STC.
FR-07	Enrollment	Validate enrollment token and CSR, then issue an STC-signed device certificate.
FR-08	Server	Expose /enroll, /clear, and /report endpoints for enrollment and invoice processing.
FR-09	Server	Decode Base64 invoice payloads and validate UBL 2.1 XML structure.
FR-10	Server	Verify invoice hash, XAdES references, signature value, certificate binding, ICV, and PIH.
FR-11	Server	Store accepted invoices, device state, taxpayer records, and audit logs in PostgreSQL.
FR-12	Server	Return a cleared invoice for clearance mode and an acknowledgment for reporting mode.

3.7 Non-Functional Requirements

Table 3.3: Non-Functional Requirements.

Quality Attribute	Requirement
Security	Protect invoice integrity and taxpayer identity through HTTPS, device enrollment, certificates, signatures, and validation checks.
Integrity	Detect invoice modification using canonicalization, SHA-256 hashes, signatures, ICV, and PIH chain continuity.
Auditability	Maintain persistent invoice records, timestamps, device records, and logs for future investigation.
Interoperability	Use structured UBL 2.1 XML to support machine-readable invoice exchange.
Reliability	Return valid responses for valid invoices, Reject invalid XML, wrong hashes, invalid signatures, broken certificate references, or broken chains.
Scalability	Separate client, backend, and database responsibilities so the prototype can evolve toward larger deployment.
Maintainability	Use layered client design and modular backend services to simplify development and explanation.
Usability	Hide cryptographic and XML details from the taxpayer user while preserving compliance behavior internally.

3.8 System Architecture

The architecture is divided into a taxpayer layer, compliance layer, and persistence layer. The taxpayer layer is represented by the Flutter client. The

compliance layer is represented by the Rust/Actix STC server. The persistence layer includes client-side SQLite and server-side PostgreSQL. The system also depends on cryptographic and XML tooling such as OpenSSL and stc-cli for signing and canonicalization tasks.

Taxpayer Client Layered Architecture

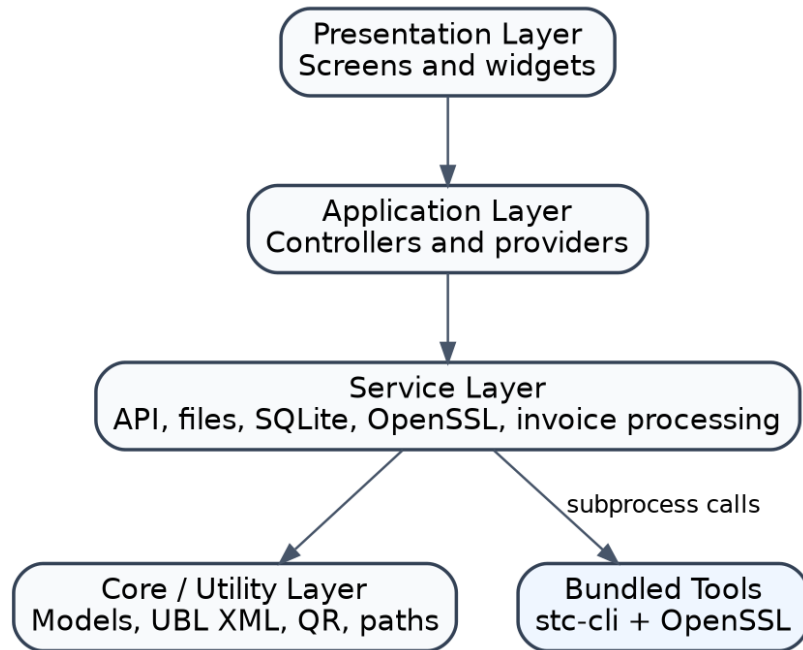


Fig 3.3: Taxpayer Client Layered Architecture.

3.8.1 Taxpayer Client Application

The taxpayer client is a cross-platform Flutter application. It is responsible for invoice data entry, UBL XML construction, XML canonicalization, invoice hashing, XAdES-style signing, QR generation, local invoice history, sandbox testing, and API submission. Its layered design separates presentation screens, application controllers/providers, service classes, and core utility models.

3.8.2 STC Compliance Server

The STC compliance server is implemented as a Rust/Actix backend. It exposes REST endpoints for onboarding, enrollment, clearance, and reporting.

Its main responsibility is to validate taxpayer devices and submitted invoices before storing them. It validates invoice structure, verifies hashes and signatures, checks certificate binding, maintains ICV and PIH chain state, and produces JSON tracing logs for observability.

3.8.3 Persistence Layer

The persistence layer is split into local and backend storage. SQLite is used locally by the taxpayer client to maintain invoice history and chain continuity. PostgreSQL is used on the server to store taxpayers, enrolled devices, CSR challenges, invoices, and authority-side chain state. This separation allows the client to continue managing local invoice order while the authority server maintains the official accepted records.

3.9 System Modules

3.9.1 Onboarding Module

The onboarding module represents the administrative entry point for a taxpayer. It assumes that a registered taxpayer has basic credentials, such as a Taxpayer Identification Number (TIN) and password. After login, the taxpayer can access services such as requesting an enrollment token for a new device.

3.9.2 Enrollment Module

The enrollment module establishes trust between a taxpayer device and STC. The client generates a private key locally, creates a DER-encoded CSR containing taxpayer and device identity data, Base64-encodes the CSR, and submits it with an enrollment token. The server validates the token, verifies the CSR signature, confirms taxpayer identity, issues an STC-signed certificate, stores the device record, and initializes the device chain state.

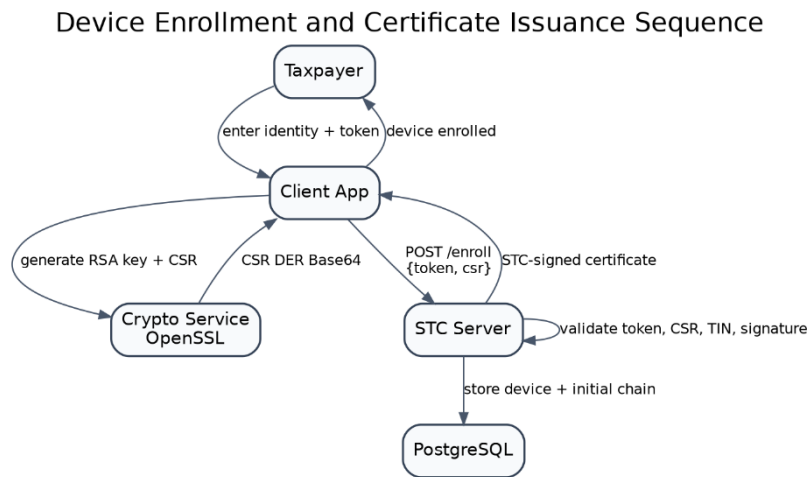


Fig 3.4: Device Enrollment and Certificate Issuance Sequence.

3.9.3 Invoice Generation and Preparation Module

The invoice generation module converts user-entered invoice data into a structured UBL 2.1 XML document. The generated invoice includes supplier and customer data, invoice lines, tax totals, legal monetary totals, UUID, invoice number, ICV, PIH, QR placeholder, and signature placeholder. The preparation module then canonicalizes the XML, calculates the hash, signs the invoice, generates QR data, and prepares the final signed XML.

The module also uses the ICV and PIH values to maintain invoice sequencing and support later audit checks. Before signing, XML canonicalization is applied so that formatting differences do not change the invoice hash. The generated hash is then used in the signing process and in the submission request to the backend. This makes the module responsible not only for creating the invoice document, but also for preparing it for compliance validation before submission.

Invoice Preparation Pipeline

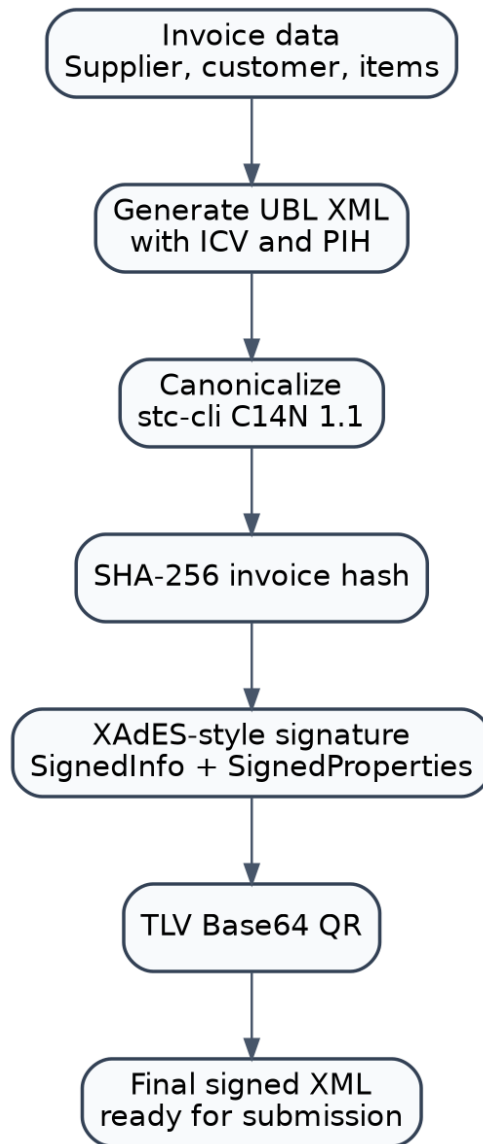


Fig 3.5: Invoice Preparation Pipeline.

3.9.4 Canonicalization Module

XML canonicalization is required because equivalent XML documents can differ in whitespace, namespace order, attribute order, or serialization format. The client uses the Rust-based stc-cli tool to apply Canonical XML 1.1 without comments before hashing and signing. This ensures that signature and hash verification depend on deterministic XML bytes rather than visual formatting.

3.9.5 Digital Signature and QR Module

The signature module follows an XAdES-style process. It hashes the canonicalized invoice, prepares SignedProperties, hashes SignedProperties, builds SignedInfo, signs SignedInfo using the taxpayer private key, and injects the resulting signature into the invoice. The QR module generates a TLV Base64 string containing verification data such as seller name, tax number, issue date, totals, invoice hash, signature, and certificate data.

3.9.6 Clearance, Reporting, Validation, and Audit Modules

The clearance module handles invoices submitted to the clearance endpoint and returns a cleared invoice when validation succeeds. This mode is used when the invoice must be approved by the STC server before it is considered accepted. After successful validation, the backend can process the invoice, update the invoice chain state, store the record, and return the cleared invoice in the response.

The reporting module handles invoices submitted to the reporting endpoint and returns an acknowledgment instead of a cleared invoice. This mode is suitable for invoices that are reported to STC for storage, tracking, and later auditing. Both clearance and reporting use the same standard submission body, which includes the invoice UUID, invoice hash, and Base64-encoded signed XML invoice.

The validation module is shared by both modes and performs the core compliance checks. It verifies the UBL XML structure, invoice hash, XAdES-style signature references, certificate binding, device enrollment status, ICV sequence, and PIH chain continuity. If any validation step fails, the invoice is rejected or flagged as invalid.

The audit/logging module records important system events such as enrollment attempts, validation failures, clearance requests, reporting submissions, and successful invoice processing.

Clearance Workflow

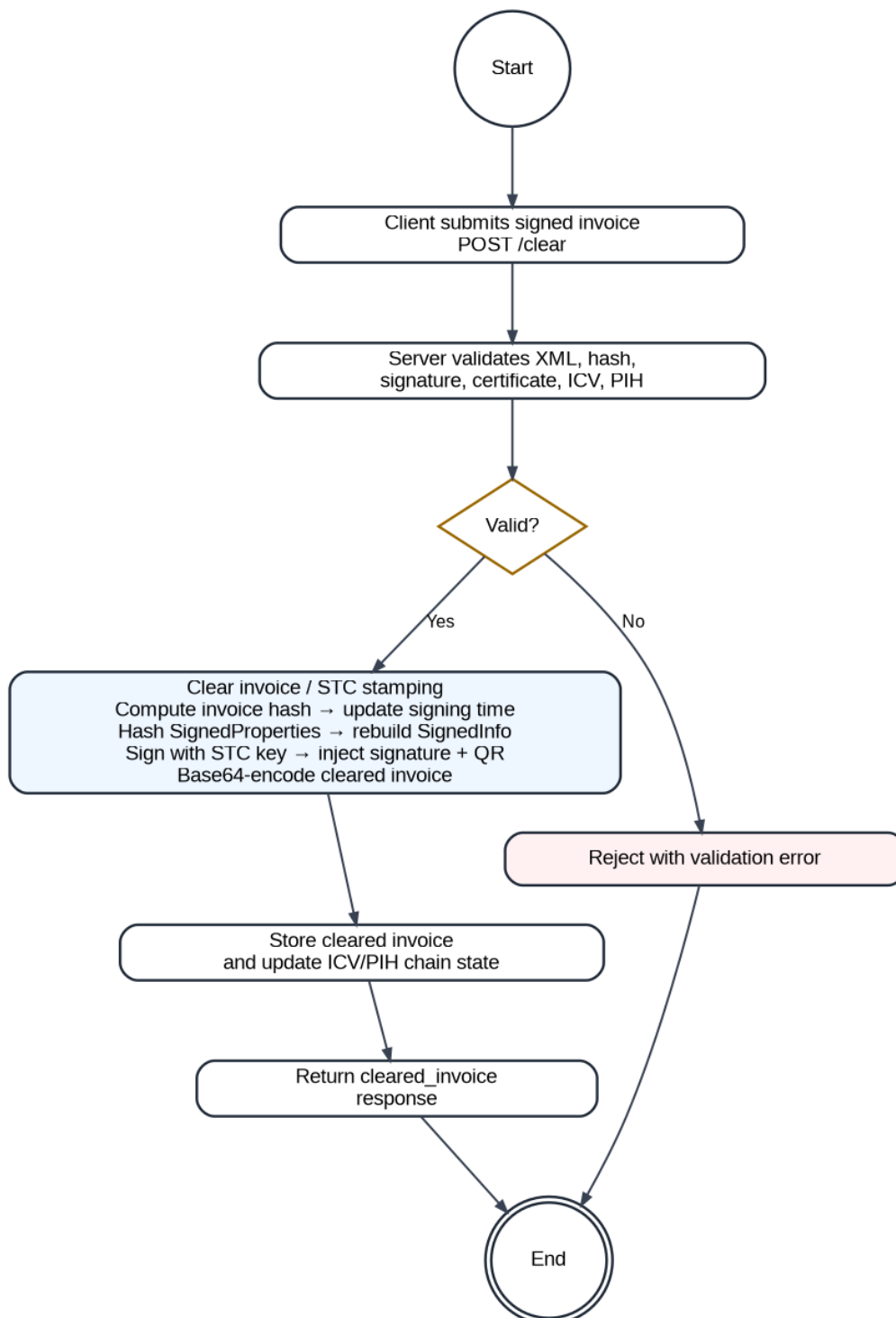


Fig 3.6: Clearance Workflow.

Reporting Workflow

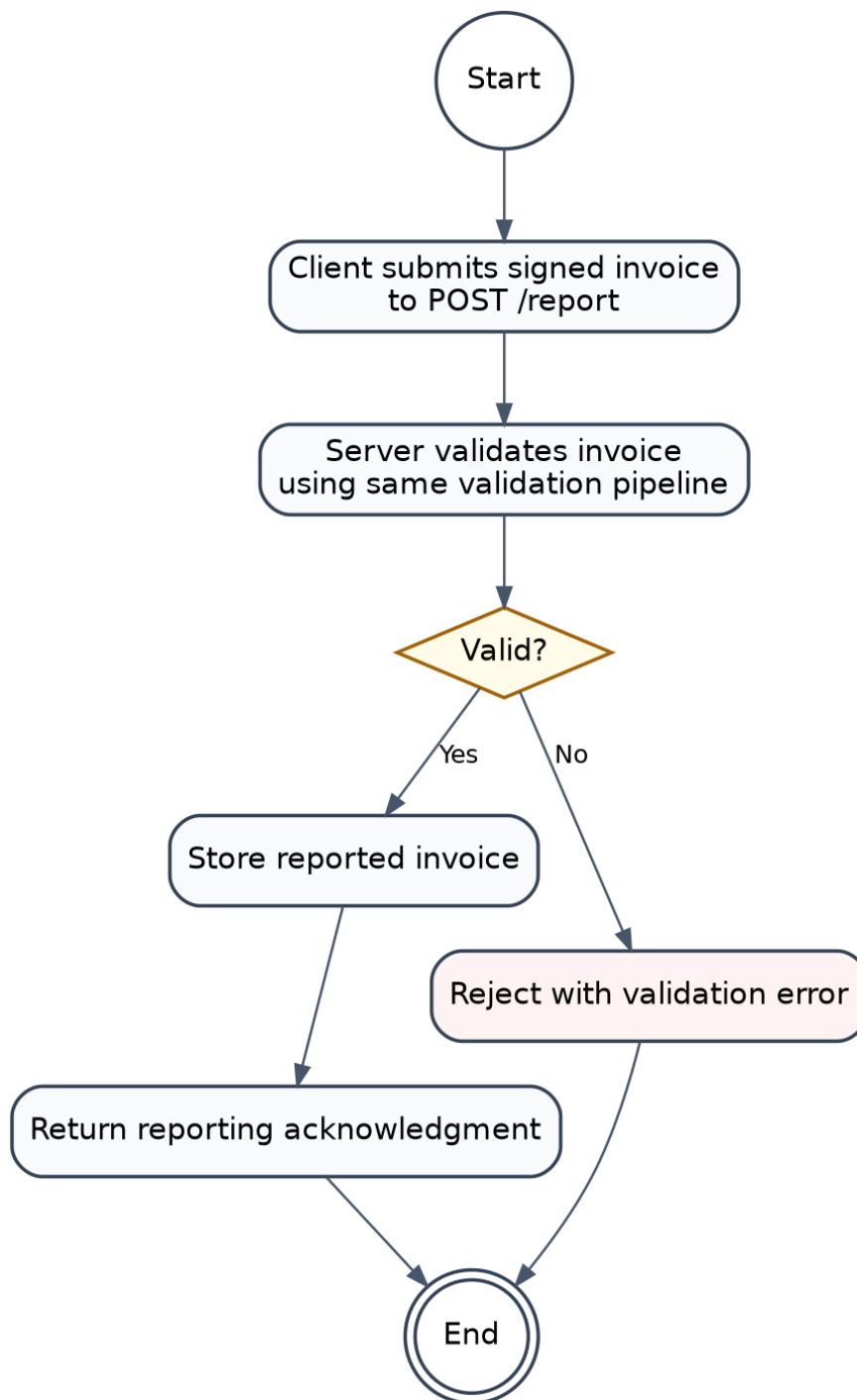


Fig 3.7: Reporting Workflow.

Backend Invoice Validation Pipeline

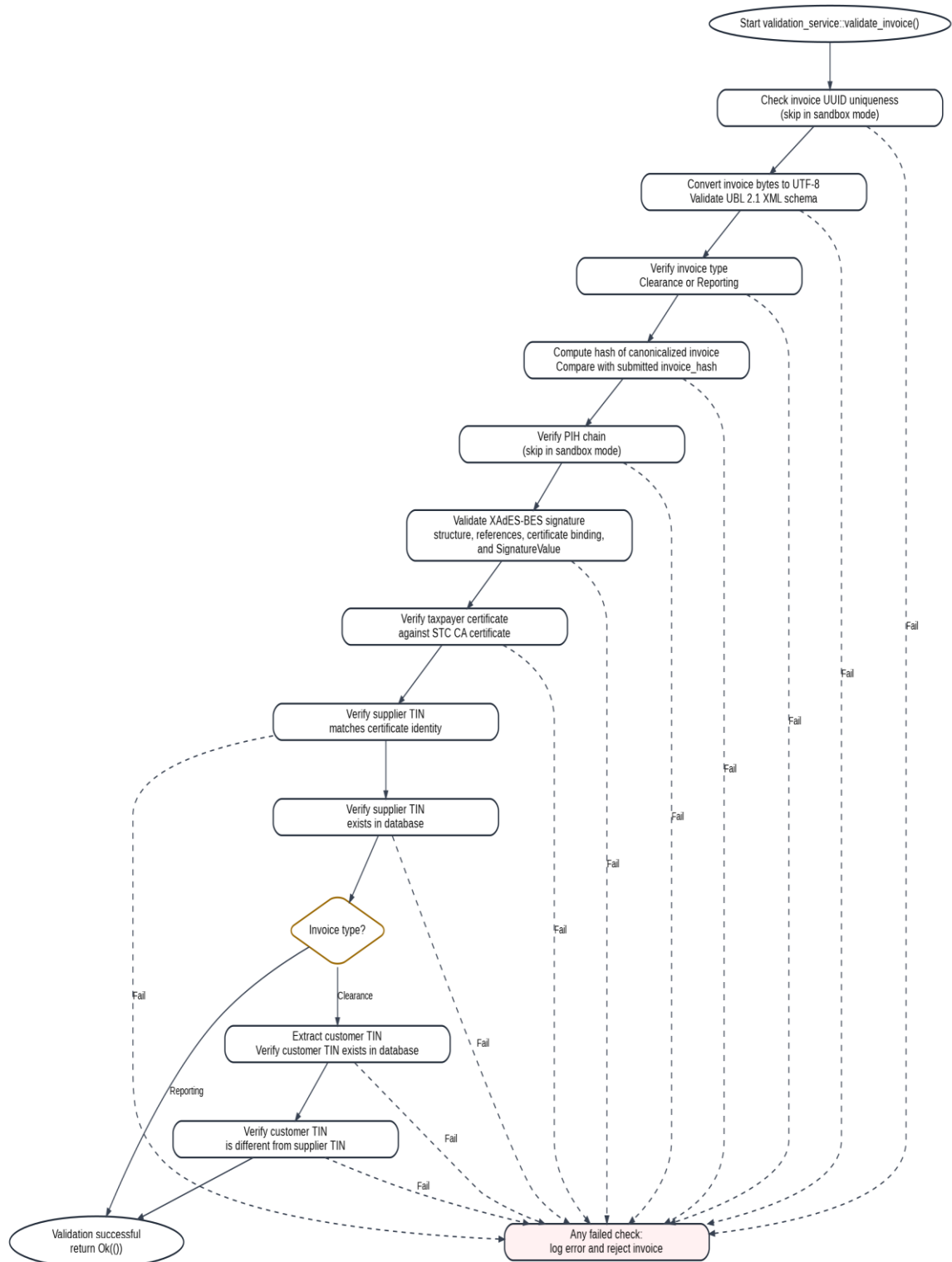


Fig 3.8: Backend Invoice Validation Pipeline.

3.10 Use Case Model

Table 3.4: Main Use Cases.

Use Case	Primary Actor	Preconditions	Main Result
Enroll Taxpayer Device	Taxpayer	Taxpayer has a valid enrollment token.	Device receives an STC-signed certificate and becomes able to sign invoices.
Generate Invoice	Taxpayer	Invoice page is open and required invoice data is available.	Unsigned UBL invoice is generated with invoice number, UUID, ICV, and PIH.
Sign Invoice	Taxpayer Device	Device has private key and certificate.	Signed XML invoice is created with XAdES-style signature and QR data.
Submit for Clearance	Taxpayer	Signed invoice exists.	Server validates and returns cleared_invoice if accepted.
Submit for Reporting	Taxpayer	Signed invoice exists.	Server validates, stores, and acknowledges reported invoice.
Validate Submitted Invoice	STC Server	Invoice submission is received.	Invoice is accepted and stored or rejected with validation error.

3.11 Database Design

The database design supports both taxpayer-side continuity and authority-side auditability. The local SQLite database stores invoice history and the latest chain information used by the client. The PostgreSQL backend stores

official taxpayer, device, enrollment, and invoice records. The logical backend entities are taxpayers, devices, CSR challenges, invoices, and audit logs.

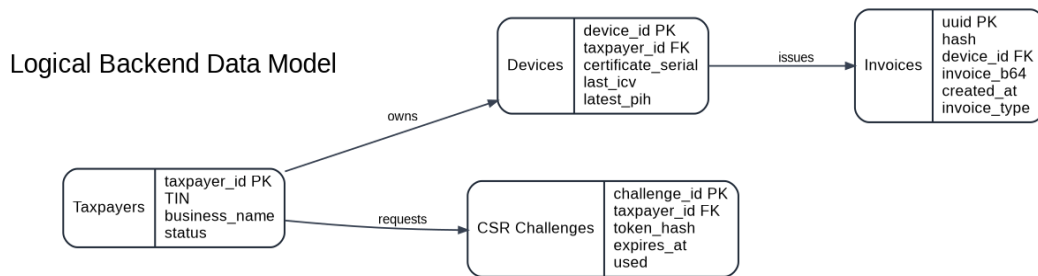


Fig 3.9: Logical Entity Relationship Diagram.

Table 3.5: Main Data Entities.

Entity	Purpose	Key Fields
Taxpayers	Stores registered taxpayer/business records.	TIN, name, address, created_at
Devices	Stores enrolled taxpayer devices and certificate state.	device_uuid, tin, current_icv, latest_pih, is_active, onboarded_at
CSR Challenges	Stores enrollment challenge/token records.	challenge_id, company_id, token_hash, expires_at, used_at
Invoices	Stores accepted clearance/reporting submissions.	uuid, hash, device_id, invoice_b64, created_at, invoice_type

3.12 API Design

The API follows REST principles. The client sends JSON requests to the backend, and signed XML invoices are Base64-encoded inside the JSON body. This simplifies transport and avoids raw XML handling issues in the HTTP request body.

Table 3.6: Main API Endpoints.

Endpoint	Method	Purpose	Main Request/Response
/enroll	POST	Enroll a taxpayer device using a CSR and token.	Request: token, csr. Response: issued certificate.
/clear	POST	Submit a signed invoice for real-time clearance.	Request: uuid, invoice_hash, invoice. Response: cleared_invoice.
/report	POST	Submit a signed invoice for reporting and storage.	Request: uuid, invoice_hash, invoice. Response: acknowledgment.
X-Sandbox-Mode	Header	Mark a request as sandbox/test submission.	Value: true for sandbox behavior.

The standard invoice submission body contains three values: uuid, invoice_hash, and invoice. The uuid identifies the invoice, invoice_hash carries the Base64 SHA-256 hash of the canonicalized invoice, and invoice carries the Base64-encoded signed XML invoice.

To standardize communication between the taxpayer client and the backend, all API endpoints use a unified JSON response structure containing a success indicator, a message, and optional response data. This allows the client to handle enrollment, clearance, reporting, and validation errors in the same way without needing different parsing logic for each endpoint. For invoice submission, the request body is also standardized around uuid, invoice_hash, and invoice, where the invoice XML is transported as Base64 text. Error responses should identify the specific validation or processing issue, such as an invalid hash, missing field, expired token, failed certificate verification, or invalid invoice chain. However, they must not expose internal implementation

details such as stack traces, database errors, server paths, cryptographic library internals, or configuration values. This design improves reliability, simplifies client-side error handling, and keeps the API consistent and secure across the main STC workflows.

3.13 Security Design

Security is central to the design because invoices are legally and financially significant documents. The system combines secure communication, device enrollment, local private-key generation, digital signatures, invoice hash verification, certificate binding, ICV/PIH chain validation, and audit logging.

The private key is generated and stored locally on the taxpayer device. The STC server receives the CSR and issues a certificate, but it never receives the taxpayer private key. When invoices are submitted, the server uses the embedded certificate and stored enrollment records to verify that the invoice was signed by an enrolled taxpayer device. The invoice hash and signature checks detect content modification, while ICV and PIH checks help detect missing, reordered or duplicated invoices.

3.14 Adaptation to Sudanese Infrastructure

The prototype is designed for gradual adoption within Sudan's developing digital infrastructure. It avoids assumptions of immediate nationwide integration with banking, customs, commercial registry, or large-scale analytics systems. Instead, it demonstrates the core compliance workflow using a lightweight client, REST APIs, local invoice preparation, local SQLite history, server-side PostgreSQL persistence, Docker-ready backend deployment, and sandbox testing.

Sandbox mode is especially important because it allows taxpayers, developers, and evaluators to test enrollment and invoice submission without affecting official production records. This supports training, onboarding, university demonstration, and phased future deployment.

3.15 Deployment Design

The deployment design separates taxpayer devices from the STC server environment. The Flutter client runs on taxpayer devices and includes the required local tools. The backend runs as a web service and connects to PostgreSQL. Docker may be used to package the server consistently with its dependencies, which improves portability between development, testing, and hosted environments.

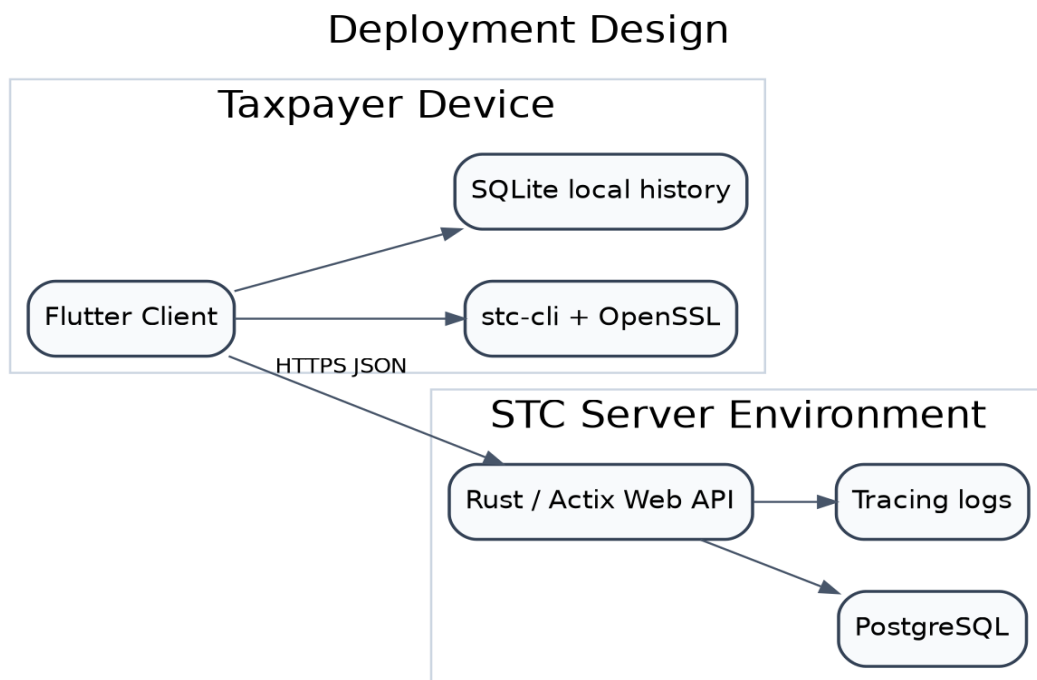


Fig 3.10: Deployment Diagram.

3.16 Prototype Limitations

Although the design follows realistic electronic invoicing concepts, the system remains a prototype. It does not represent a complete nationwide production rollout. It does not include banking integration, customs integration, commercial registry integration, advanced fraud analytics, large-scale taxpayer onboarding, a full STC officer dashboard, production-grade private-key storage, full role-based administration, or a complete observability stack. These limitations define the research scope and identify clear directions for future work.

Chapter Four

System Implementation

4.1 Introduction

This chapter presents the implementation of the Digital E-Invoicing and Tax Compliance Platform Prototype for the Sudan Taxation Chamber (STC). Chapter 3 explained the system analysis, design decisions, architecture, workflows, database model, API structure, and security design. Therefore, this chapter focuses on the practical implementation: the tools used, the implemented client and server functions, the working screens, API execution, logs, deployment, and performance observations.

4.2 Development Tools and Technologies

The prototype was implemented using a small set of tools selected to support cross-platform client development, high-performance backend validation, structured XML processing, cryptographic operations, database persistence, and cloud-ready deployment.

4.2.1 Main Implementation Tools and Their Use in the Prototype

- **Client Application:** Flutter and Dart were used to build the taxpayer-side application, including screens, controllers, service orchestration, local state management, and cross-platform user interface.
- **Backend Service:** Rust and Actix Web were used to implement the STC-side REST API for onboarding, enrollment, clearance, reporting, validation, and structured JSON responses.
- **Database:** PostgreSQL was used to store authority-side taxpayer, device, and invoice records, while SQLite was used to store local client invoice history and invoice chain state.
- **Cryptography:** OpenSSL, SHA-256, RSA, and X.509 certificates were used for private key generation, CSR creation, certificate issuance, signature generation, and signature verification.

- **XML Processing:** UBL 2.1 schemas, stc-cli, and XML C14N 1.1 were used for invoice XML generation, schema validation, canonicalization, deterministic hashing, and signing support.
- **Networking:** REST APIs and HTTPS-ready JSON transport were used for client-server communication, with signed XML invoices encoded in Base64 inside JSON request bodies.
- **Deployment and Testing:** Docker, Render hosting, GitHub Actions, and logging tools were used for backend packaging, hosted demonstration, automated checks, runtime logging, and prototype profiling.

4.3 Implementation Setup

The implementation is divided into three practical parts: the Flutter taxpayer client, the Rust/Actix STC backend, and the persistence layer. The client contains the user interface, invoice preparation services, local file management, local SQLite records, and API integration. The backend contains REST routes, service modules, validation logic, certificate handling, PostgreSQL access, and JSON tracing logs. [23], [24].

The client starts by initializing Flutter bindings, SQLite FFI support, tool extraction, and global providers. The bundled tools are prepared before the application interface starts, because invoice canonicalization and signing depend on local command-line tools. This ensures that the required native executables are available before invoice generation or certificate enrollment is attempted.

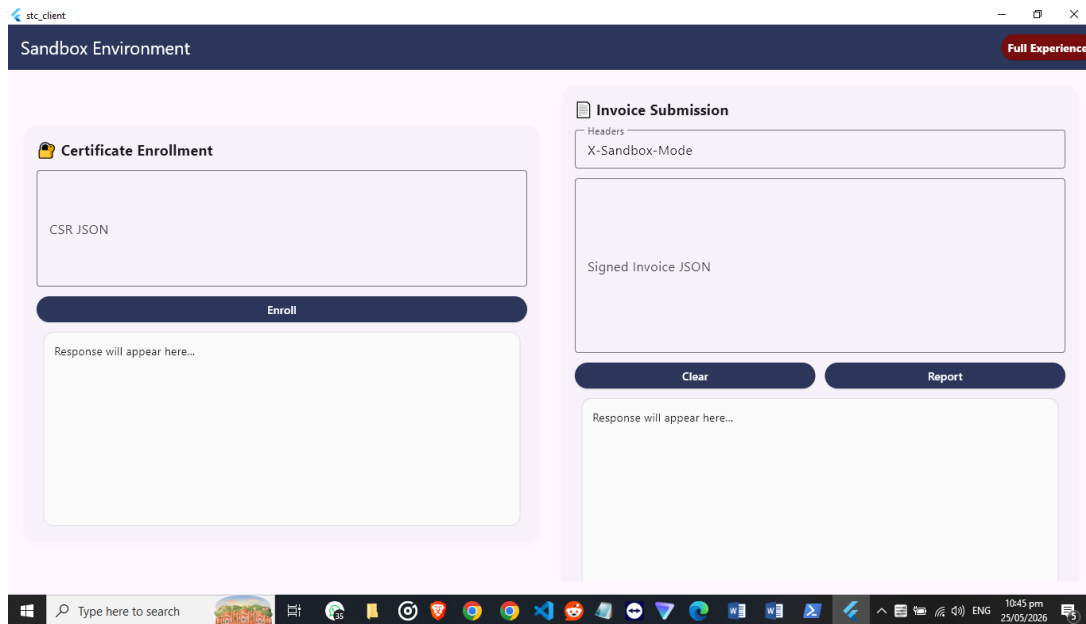


Fig 4.1: Taxpayer client startup screen.

The backend is implemented as an Actix Web service. At startup, it loads configuration, initializes database connectivity, registers the API routes, and starts the HTTP server. The main implemented routes used by the prototype are /onboard, /enroll, /clear, and /report.

4.4 Onboarding Implementation

Onboarding is implemented as the entry step that allows an already registered taxpayer to request a one-time enrollment token for a new device. The prototype assumes that a taxpayer record already exists in the system and that the taxpayer can access a simple portal using a TIN and password.

4.4.1 Taxpayer Login Page

The taxpayer login page collects the taxpayer identification number and password. After successful login, the taxpayer is taken to a home page that exposes the main administrative action: generating a one-time enrollment token for a new device. This token is later used by the device enrollment process.

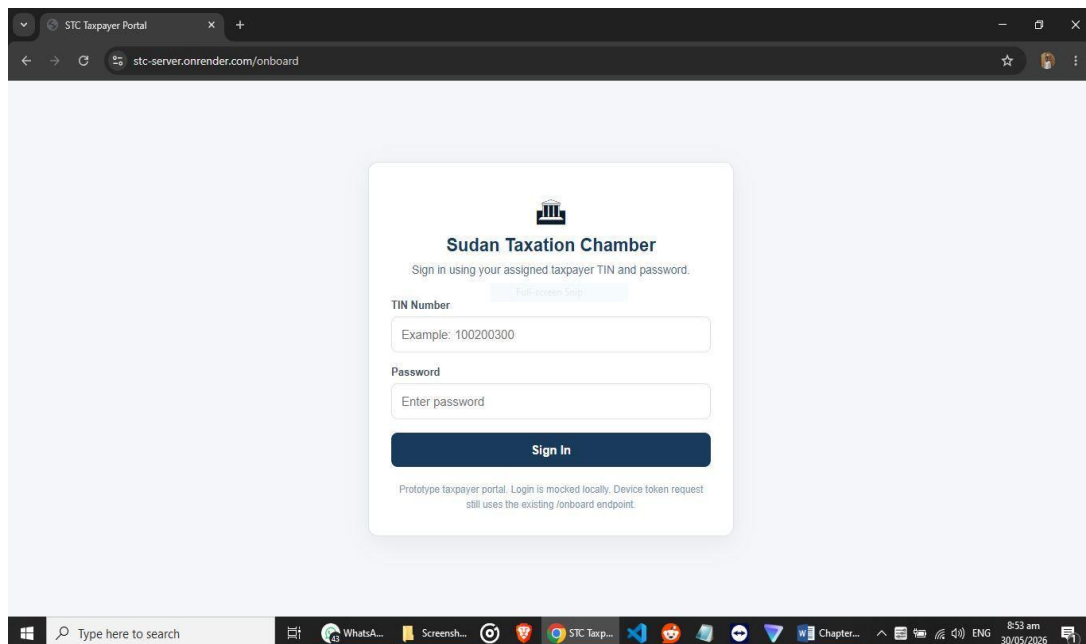


Fig 4.2: Taxpayer login screen.

4.4.2 Enrollment Token Generation

After login, the taxpayer can request an enrollment token. The backend generates a one-time token, stores only a hashed challenge value, assigns an expiry time, and returns the plain token once to the taxpayer interface. This token is then copied into the client enrollment screen when the device submits its CSR.

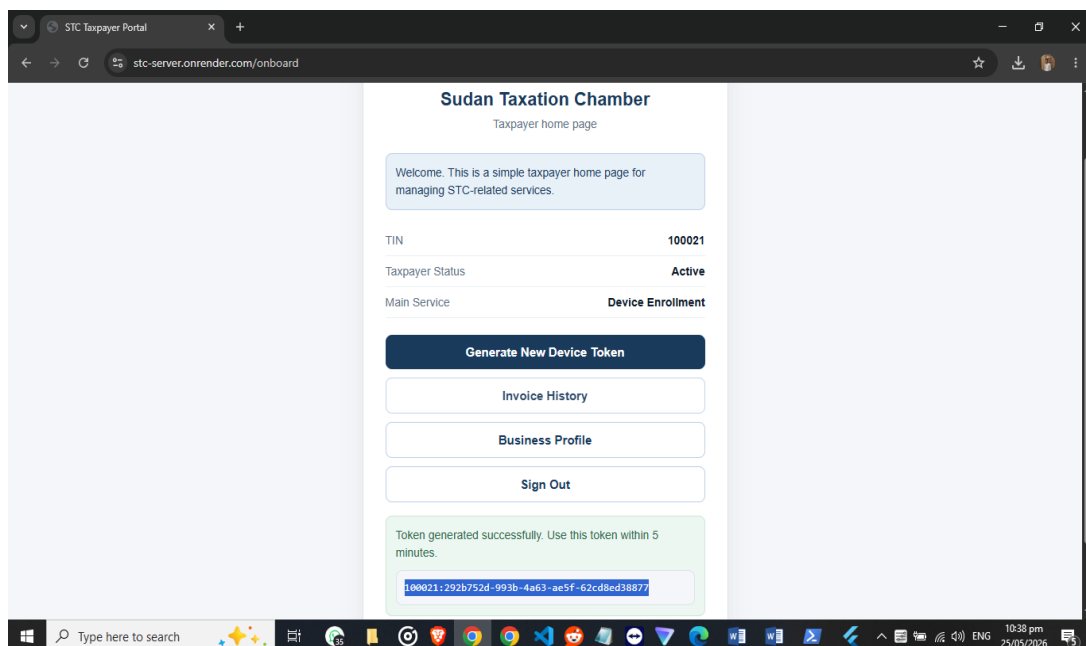


Fig 4.3: One-time enrollment token generation.

4.4.3 Token Expiry and Usage Control

Token-based enrollment is protected using expiration, single-use enforcement, and periodic cleanup. When a taxpayer device submits an enrollment request, the backend checks whether the token hash exists, whether the token is still valid, and whether it has already been consumed. Each token expires after five minutes, limiting the time window in which it can be used. After successful enrollment, the token is marked as used, ensuring that it cannot be reused to enroll another device. Furthermore, the backend runs a scheduled task every hour to remove expired tokens from storage. This reduces obsolete token records and strengthens the enrollment process by combining authorization control with automatic lifecycle management.

4.5 Enrollment Implementation

Enrollment establishes trust between the taxpayer device and the STC server. The implementation keeps the private key on the taxpayer device and sends only the CSR and token to the backend. The backend verifies the enrollment request, issues a certificate, and creates an enrolled device record.

4.5.1 Local Private Key and CSR Generation

The client enrollment screen collects certificate subject information such as common name, organization name, country, locality, and serial number. The enrollment service calls OpenSSL to generate an RSA private key and a DER-encoded certificate signing request. The private key is stored locally and is not sent to the server.

stc_client

← Certificate Generator

Private Key

```
-----BEGIN PRIVATE KEY-----
MIIEvAIBADANBgkqhkiG9w0BAQEFAASC8YwggSiAgEAAQBAQDMuY2Pzdlufhv1
p+dSEGZD3DrgUNIMTjvAWwpxfpuNP5Pead0aeSDFQcY17RfVajW9YUwXhW4oka
KiHmbLjg8oI7osNUePTOSqstyd2dwC6SUPZaphkbP/7+pkT85y3xnZ7RQMUBMIF
c+Xlpz5Fcg5xIV8mqzx0I94S/WOUZqxZjGosFfid10eIEC110VURQ7nP176gQd8
SpsoTn6Qey6ni2Yneg/inm6YgUBM9PDxAvGpIQjEU0dGQR8f0RbMV4h11eCIKQT/
rN1ILMFKrC4M4Bhyn117n8fB8dKc7k4C6f8IDn16u10s5awwK3N07hnnnfnf45C
```

Generated CSR

```
MIIC3zCCAccCAQAwZkxZzAvBGNVBAAMMDk15LkNvbXh8bnkuY29tMQ8wDQYDVQQKDAVxMDAwMjExCj
zAIBGNVBAAMMAkUUMQswCQYDVQQGEwJTRDRERMA8GA1UECAwIS2hhcnRvdW0kETAPBgNVBACMcEtoYXU
0b3V1MS0wkwYDVQQFeyRmZmUwZjllMio2Y2FkLTQ5MzktYmZjZjAxNDc0NDkwggEIMA0GC
SgGSlb3DQEBAQUAA4IBDwAwggEKAoIBAQMdmuY2Pzdlufhv1p+dSEGZD3DrgUNIMTjvAWwpxfpuNP5P
ead0aeSDFQcY17RfVajW9YUwXhW4okaKiHmbLjg8oI7osNUePTOSqstyd2dwC6SUPZaphkbP/
7+pkT85y3xnZ7RQMUBMIFc+Xlpz5Fcg5xIV8mqzx0I94S/
WOUZqxZjGosFfid10eIEC110VURQ7nP176gQd8SpsoTn6Qey6ni2Yneg/inm6YgUBM9PDxAvGpIQjEU0dGQR8f0RbMV4h11eCIKQT/
```

CSR Subject Information

Common Name (CN) Organization Name (ON)

Organizational Unit (OU) Country (C)

State (ST) Locality (L)

Serial Number

[Generate New CSR & Key](#)

Enrollment Token

Insert Token

[Generate Certificate](#)

Received Certificate

— Empty —

[Go to Invoice Page](#)

Fig 4.4: Device enrollment form.

4.5.2 CSR Submission to STC Server

After the CSR is generated, the client Base64-encodes it and sends it to the /enroll endpoint together with the one-time token. The API body is simple and contains only the token and CSR because identity proof is carried by the token and the CSR subject data.

4.5.3 CSR Verification and Certificate Issuance

On the server side, the enrollment implementation decodes and parses the CSR, extracts the taxpayer identity and device UUID from the csr SERIALNUMBER field, checks the token hash, verifies that the token is not expired or used, verifies the CSR signature, checks that the taxpayer exists, signs the CSR using the STC certificate authority logic, creates the device record, initializes ICV and PIH values, marks the token as used, and returns the issued device certificate.

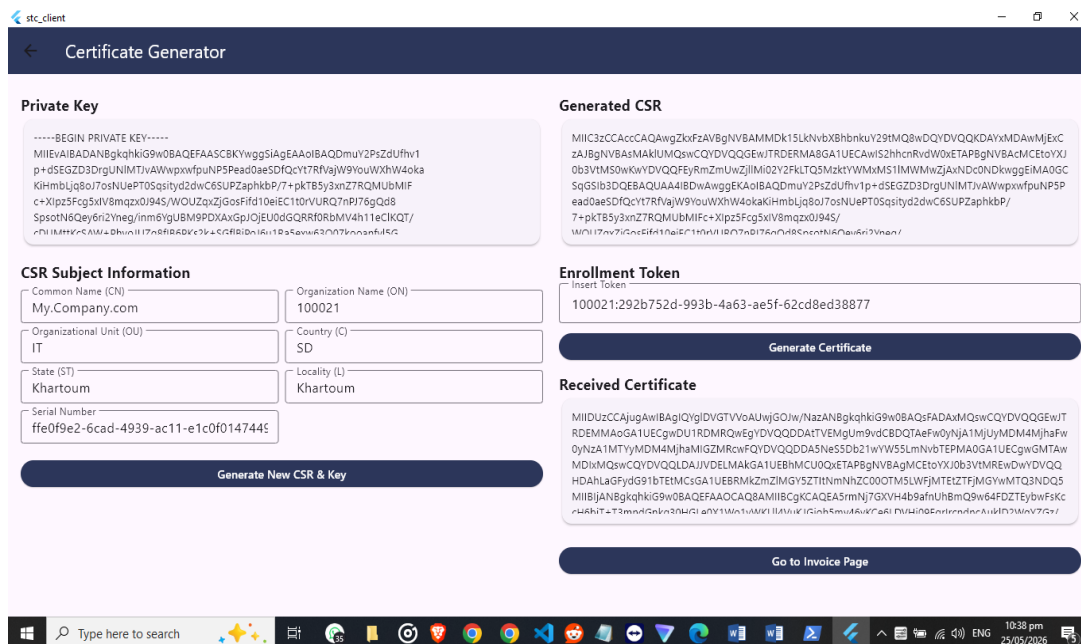


Fig 4.5: Certificate enrollment success screen.

4.6 Invoice Generation and Signing Implementation

The invoice generation and signing implementation is the largest client-side workflow. It converts user-entered invoice data into UBL 2.1 XML, prepares the invoice for deterministic hashing, signs the invoice using the taxpayer private key, and prepares the final signed XML for submission.

4.6.1 UBL 2.1 Invoice Generation

The invoice page collects supplier, customer, and invoice line data. The client then builds a UBL 2.1 invoice containing the invoice number, UUID, issue date and time, supplier party, customer party, invoice lines, tax totals, legal monetary totals, ICV, PIH, QR placeholder, and signature placeholder. Default demonstration values are used where appropriate to support a faster project demo.

STC Invoice Generator

Invoice Information

Invoice Number: 895f3338-fbba-4ba9-8437-930453d8cb58
 Invoice Date: 2026-05-30
 Invoice Type: 380
 Currency Code: SDG

Supplier Information

Supplier Name: My Supplier
 Supplier TIN: 100021
 Street Name: Baladyia st
 City: Khartoum
 Country: SD
 Phone: +249912345678
 Email: supplier@example.cor

Customer Information

Customer Name: Default Customer
 Customer TIN: 100011
 Street Name: Baladyia st
 City: Omdurman
 Country: SD
 Phone: +249911111111
 Email: customer@example.cc

Invoice Items

Item Name	Qty	Price	Tax (%)
Laptop	2	1500.0	
Dell			15.0

Totals

Subtotal: 3000.00 SDG
 Tax Total: 450.00 SDG
 Grand Total: 3450.00 SDG

Preview Mode (Toggle: Off) **XML / JSON**

XML will appear here

Response

Server response will appear here...

Generate B2C Invoice Endpoint: <https://stc-server.onrender.com/report>
Generate B2B Invoice Endpoint: <https://stc-server.onrender.com/clear>
Clear Invoice Endpoint: <https://stc-server.onrender.com/clear>
Report Invoice Endpoint: <https://stc-server.onrender.com/report>

Fig 4.6: Invoice data entry screen.

4.6.2 Canonicalization Using stc-cli

After the unsigned invoice XML is generated, the client writes it to a working file and calls the bundled stc-cli executable. The tool receives input and output XML paths and applies Canonical XML 1.1 without comments. This produces deterministic XML bytes for hashing and signing.

The command output is checked by the client service. If canonicalization fails, invoice preparation is stopped and an error is shown rather than continuing with an invalid hash or signature.

4.6.3 Invoice Hashing and XAdES Signing

The signing service calculates a SHA-256 hash of the canonicalized invoice, prepares SignedProperties, calculates the SignedProperties hash, builds SignedInfo, signs SignedInfo using RSA-SHA256 through OpenSSL, and injects the resulting signature into the XML invoice. The implementation also includes certificate information so that the backend can verify the taxpayer device certificate during validation.

STC Invoice Generator

Invoice Information

Invoice Number: 895f338-fbba-4ba9-8437-930453d8cb58
 Invoice Date: 2026-05-30
 Invoice Type: 380
 Currency Code: SDG

Supplier Information

Supplier Name: My Supplier
 Supplier TIN: 100021
 Street Name: Baladyia st
 City: Khartoum
 Country: SD
 Phone: +249912345678
 Email: supplier@example.cor

Customer Information

Customer Name: Default Customer
 Customer TIN: 100011
 Street Name: Baladyia st
 City: Omdurman
 Country: SD
 Phone: +249911111111
 Email: customer@example.cc

Invoice Items

Item Name	Qty	Price	Tax (%)
Laptop	2	1500.0	15.0
Dell			

Totals

Subtotal	3000.00 SDG
Tax Total	450.00 SDG
Grand Total	3450.00 SDG

Preview Mode (XML / JSON)

```
<?xml version="1.0" encoding="UTF-8"?>
<Invoice xmlns="urn:oasis:names:specification:ubl:schema:xsd:Invoice-2"
xmlns:cac="urn:oasis:names:specification:ubl:schema:xsd:CommonAggregateComponents-2"
xmlns:cbc="urn:oasis:names:specification:ubl:schema:xsd:CommonBasicComponents-2"
xmlns:ds="http://www.w3.org/2000/09/xmldsig#"
xmlns:ext="urn:oasis:names:specification:ubl:schema:xsd:CommonExtensionComponents-2"
xmlns:sac="urn:oasis:names:specification:ubl:schema:xsd:SignatureAggregateComponents-2"
xmlns:sig="urn:oasis:names:specification:ubl:schema:xsd:CommonSignatureComponents-2"
xmlns:xades="http://uri.etsi.org/01903/v1.3.2#">
  <ext:UBLExtensions>
    <ext:UBLExtension>
      <ext:ExtensionURI>urn:oasis:names:specification:ubl:schema:xsd:SignatureAggregateComponents-2
      <ext:ExtensionContent>
        <sig:UBLDocumentSignatures>
          <sac:SignatureInformation>
            <ds:Signature
              id="signature">
              <ds:SignedInfo>
                <ds:CanonicalizationMethod Algorithm="http://www.w3.org/2006/12/xml-c14n11#" />
                <ds:SignatureMethod Algorithm="http://www.w3.org/2001/04/xmldsig-more#rsa-sha256" />
                <ds:Reference URI="">
                  <ds:Transforms>
                    <ds:Transform Algorithm="http://
```

Response

Server response will appear here...

Generate B2C Invoice | **Generate B2B Invoice** | **Clear Invoice** | **Report Invoice**

Endpoint: <https://stc-server.onrender.com/report> | <https://stc-server.onrender.com/clear> | <https://stc-server.onrender.com/clear> | <https://stc-server.onrender.com/report>

Fig 4.7: Signed invoice generation result.

4.6.4 QR Code Generation

The client generates a TLV Base64 QR string containing key verification fields such as seller name, tax number, issue date and time, invoice total, VAT total, invoice hash, digital signature, and certificate information. The QR value is inserted into the invoice XML.

4.6.5 ICV and PIH Chain Handling

The client uses local SQLite records to keep the latest invoice counter value and previous invoice hash. Before generating a new invoice, the client reads the latest local chain state, increments the ICV, and embeds the previous invoice hash as PIH. After a clearance or reporting response is processed, the local chain is updated with the accepted invoice hash.

4.7 Clearance and Reporting Implementation

Clearance and reporting are implemented as two submission modes that reuse the same signed invoice preparation model. In both modes, the client sends a JSON request containing the invoice UUID, the Base64 SHA-256 invoice hash, and the Base64-encoded signed XML invoice. The differences are

in the processing and result: clearance invoices will be validated, cleared and stored by the backend and returns a cleared invoice as a result, while reporting validates and stores and returns an acknowledgment.

4.7.1 Clearance Workflow

The clearance function is used when the invoice must be validated by STC before final acceptance. The client sends the signed invoice to /clear endpoint . The backend decodes the XML, validates the invoice, applies the server-side clearance process, stores the cleared invoice, updates the ICV and PIH chain and returns a response containing cleared_invoice.

The screenshot displays the 'STC Invoice Generator' web application. The main form is divided into several sections: 'Invoice Information' (Invoice Number, Invoice Date, Invoice Type, Currency Code), 'Supplier Information' (Supplier Name, Supplier TIN, Street Name, City, Country, Phone, Email), 'Customer Information' (Customer Name, Customer TIN, Street Name, City, Country, Phone, Email), and 'Invoice Items' (Item Name, Qty, Price, Description). A 'Totals' section shows Subtotal (3000.00 SDG), Tax Total (450.00 SDG), and Grand Total (3450.00 SDG). On the right, a 'Preview Mode' toggle is set to 'XML', showing a signed XML invoice. Below the XML, a 'Response' section displays the status code 200 and a detailed message. At the bottom, there are four buttons: 'Generate B2C Invoice', 'Generate B2B Invoice', 'Clear Invoice', and 'Report Invoice'. The 'Clear Invoice' button is highlighted.

Fig 4.8: Clearance workflow screen.

4.7.2 Reporting Workflow

The reporting function is used when the invoice is submitted for authority recording rather than returned as a cleared copy. The client sends the same request structure to /report endpoint . The backend validates the invoice using the same core validation logic, stores the invoice record, updates device chain state, and returns an acknowledgment.

STC Invoice Generator

Invoice Information

Invoice Number: 2c0cb560-5ddc-4fc5-be8c-432e0a51853c
 Invoice Date: 2026-05-30
 Invoice Type: 380
 Currency Code: SDG

Supplier Information

Supplier Name: My Supplier
 Supplier TIN: 100021
 Address: Khartoum Bahri
 City: Khartoum
 Country: SD
 Phone: +249912345678
 Email: supplier@example.com

Customer Information

Customer Name: Default Customer
 Customer TIN: 100011
 Address: Baladyia st
 City: Omdurman
 Country: SD
 Phone: +249911111111
 Email: customer@example.com

Invoice Items

Item Name	Qty	Price	Tax (%)
Laptop	2	1500.0	15.0
Dell			

Totals

Subtotal: 3000.00 SDG
 Tax Total: 450.00 SDG
 Grand Total: 3450.00 SDG

Preview Mode

XML / JSON

```
<?xml version="1.0" encoding="UTF-8"?>
<Invoice xmlns="urn:oasis:names:specification:ubl:schema:xsd:Invoice-2"
xmlns:cac="urn:oasis:names:specification:ubl:schema:xsd:CommonAggregateComponents-2"
xmlns:cbc="urn:oasis:names:specification:ubl:schema:xsd:CommonBasicComponents-2"
xmlns:ds="http://www.w3.org/2000/09/xmldsig#"
xmlns:ext="urn:oasis:names:specification:ubl:schema:xsd:CommonExtensionComponents-2"
xmlns:sac="urn:oasis:names:specification:ubl:schema:xsd:SignatureAggregateComponents-2"
xmlns:sig="urn:oasis:names:specification:ubl:schema:xsd:CommonSignatureComponents-2"
xmlns:xades="http://uri.etsi.org/01903/v1.3.2#">
  <ext:UBLExtensions>
    <ext:UBLExtension URI="urn:oasis:names:specification:ubl:dsig:enveloped:xades" />
  </ext:UBLExtensions>
  <ext:ExtensionContent>
    <sig:UBLDocumentSignatures>
      <sac:SignatureInformation>
        <ds:Signature
          id="signature">
          <ds:SignedInfo>
            <ds:CanonicalizationMethod Algorithm="http://www.w3.org/2006/12/xml-c14n11#" />
            <ds:SignatureMethod Algorithm="http://www.w3.org/2001/04/xmldsig-more#rsa-sha256" />
            <ds:Reference URI="">
              <ds:Transforms>
                <ds:Transform Algorithm="http://
```

Response

Response code: 202
 Body:
 {
 "success": true,
 "message": "Invoice reported",
 "data": null
 }

Buttons: Generate B2C Invoice, Generate B2B Invoice, Clear Invoice, Report Invoice

Endpoints:
 Generate B2C Invoice: https://stc-server.onrender.com/report
 Generate B2B Invoice: https://stc-server.onrender.com/clear
 Clear Invoice: https://stc-server.onrender.com/clear
 Report Invoice: https://stc-server.onrender.com/report

Fig 4.9: Reporting workflow screen.

4.7.3 Response Handling and Local Storage

After submission, the client parses the unified API response. In clearance mode, the returned cleared_invoice is decoded, saved locally, and processed as the accepted invoice copy. In reporting mode, the acknowledgment is used to confirm that the invoice was stored by the authority-side server; the same sent invoice is treated as the accepted invoice. In both cases, the local invoice history is updated so the next invoice can continue the ICV and PIH chain.

4.8 Server-Side Validation and Storage Implementation

The backend validation implementation is shared by clearance and reporting. The server receives the JSON body, decodes the Base64 XML invoice, parses the invoice, checks uuid for duplication, validates its UBL structure, canonicalizes relevant XML sections, verifies the submitted invoice hash, checks XAdES references, verifies the signature value against the embedded device certificate, checks certificate binding against the enrolled device record, validates ICV and PIH continuity, and stores accepted invoices in PostgreSQL.

Table 4.1: Implemented backend invoice validation steps.

Validation Step	Implemented Action
Request decoding	Read uuid, invoice_hash, and invoice from the JSON body and decode the Base64 XML payload.
UBL validation	Validate the XML invoice against the local UBL 2.1 schema files.
Hash verification	Canonicalize invoice content and compare the calculated hash with the submitted hash.
Signature verification	Verify XAdES-style references, certificate data, SignedInfo, and RSA signature value.
Certificate binding	Confirm that the Certificate is signed by the backend CA.
Chain validation	Check ICV sequence and PIH continuity for the device.
Persistence	Store accepted invoice records and update latest device chain state.

uuid	hash	device_id	invoiceb64	created_at	invoice_type
dab58cb3-1b18-...	\x5b595702bcbaccd141792ca...	900f621a-3c65-4c00...	PEludm9pY2UgeG1sbn...	2026-04-12 20:52:24.981596+00	clearance
f8aed074-d14e-...	\x8f4092836cde52977cf33bf...	900f621a-3c65-4c00...	<Invoice xmlns="ur...	2026-04-12 20:52:25.040698+00	reporting
05f71da8-74b9-...	\xa31ec2f90070ae655457ebfc...	900f621a-3c65-4c00...	PEludm9pY2UgeG1sbn...	2026-04-12 20:52:25.095403+00	clearance
48017988-1c55-...	\x9278b455882576e438ef6ec4...	900f621a-3c65-4c00...	<Invoice xmlns="ur...	2026-04-12 20:52:25.15431+00	reporting
3ccbb672-e2ba-...	\xbff17a876a5e78f9105a5fcc8...	900f621a-3c65-4c00...	PEludm9pY2UgeG1sbn...	2026-04-12 20:52:25.211511+00	clearance
61b44de1-f4e1-...	\x3cdf241e7a63deddad54433c...	900f621a-3c65-4c00...	<Invoice xmlns="ur...	2026-04-12 20:52:25.27168+00	reporting
f2e6c527-ad87-...	\x8ca949fe2429960b8f03d1ec...	900f621a-3c65-4c00...	PEludm9pY2UgeG1sbn...	2026-04-12 20:52:25.329386+00	clearance
2de7cdcb-3df1-...	\x924c9df5d2ca07574e764d62...	900f621a-3c65-4c00...	<Invoice xmlns="ur...	2026-04-12 20:52:25.386846+00	reporting
19dfb7da-d8be-...	\xc8d9815dc289aab56000368a...	900f621a-3c65-4c00...	PEludm9pY2UgeG1sbn...	2026-04-12 20:52:25.445938+00	clearance
339e4477-4a1a-...	\xbcc5a27cfe59b774ecac4ef...	900f621a-3c65-4c00...	<Invoice xmlns="ur...	2026-04-12 20:52:25.502444+00	reporting
e1d68dcf-cbd9-...	\x7fa24f32e0b3f8138e91ace3...	900f621a-3c65-4c00...	PEludm9pY2UgeG1sbn...	2026-04-12 20:52:25.551981+00	clearance
21a054fb-be75-...	\x6fd4e9114e6ed0ec5ca93cbd...	900f621a-3c65-4c00...	<Invoice xmlns="ur...	2026-04-12 20:52:25.603442+00	reporting
1226adc0-e26b-...	\x62cea59c25d99cf3650abd6f...	900f621a-3c65-4c00...	PEludm9pY2UgeG1sbn...	2026-04-12 20:52:25.65471+00	clearance
1437d334-1b1c-...	\xc6c61ad24f768eecd5b5d0f1...	900f621a-3c65-4c00...	<Invoice xmlns="ur...	2026-04-12 20:52:25.707503+00	reporting
95b2510c-1f3c-...	\xe89a71e921480c3eadeec292...	900f621a-3c65-4c00...	PEludm9pY2UgeG1sbn...	2026-04-12 20:52:25.909311+00	clearance
4f0d369f-8aeb-...	\x3ff1b81daa5b980939be0f1d...	900f621a-3c65-4c00...	<Invoice xmlns="ur...	2026-04-12 20:52:25.980227+00	reporting
08fa3100-a223-...	\xb669b42ca380ca51396cc477...	e5145f38-6d45-42a5...	PEludm9pY2UgeG1sbn...	2026-04-12 20:52:26.033273+00	clearance

Fig 4.10: Stored backend invoice records.

4.9 Sandbox mode

Sandbox mode is handled at the submission layer using the X-Sandbox-Mode header. When this header is enabled, the client sends the same clearance

or reporting request format, but the backend treats the submission as a test transaction. This allows the testing of invoice validation, hashes and signatures without requiring a fully working client-side system with working invoice uuid, ICV and PIH. This is done to help client-side developers debug their systems without requiring real invoices.

Fig 4.11: Sandbox response screen.

4.10 Auditing and Logs Implementation

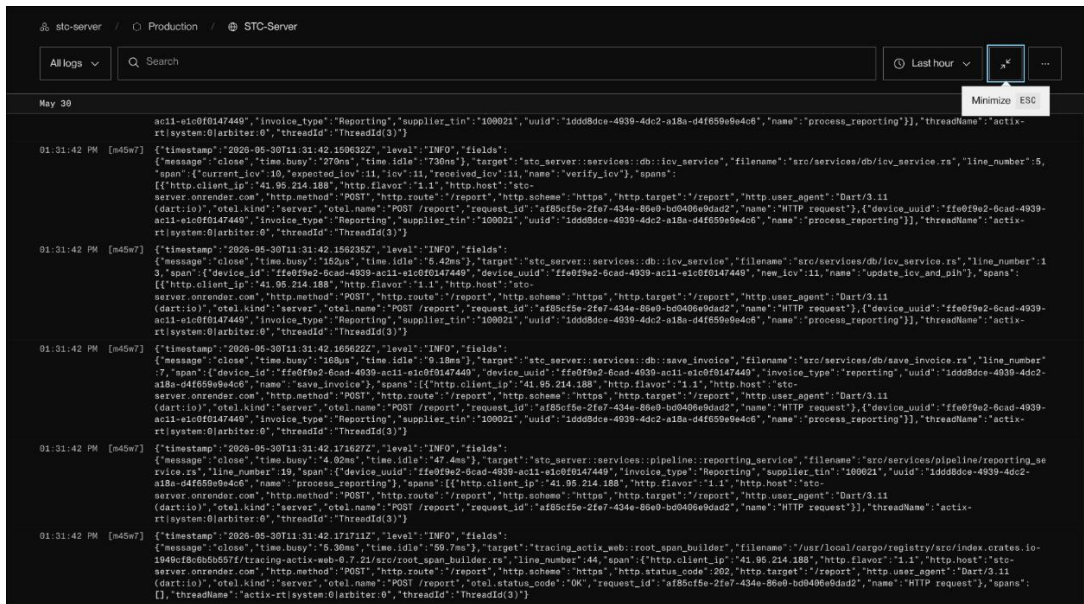


Fig 4.12: Runtime logging output.

4.11 Deployment and Performance

The backend is prepared for deployment using Docker-compatible project files and can run as a hosted web service connected to PostgreSQL. For the prototype demonstration, the server can be deployed to a cloud hosting service while the Flutter client communicates with it through the configured base URL. This deployment approach allows the system to be demonstrated as a real client-server application rather than a local-only program.

Performance is supported primarily by the Rust backend, modular validation services, database-backed persistence, and compact JSON transport. The invoice XML is transported as Base64 text inside JSON, which simplifies HTTP handling and avoids raw XML body parsing problems. Profiling artifacts and runtime logs can be used to inspect bottlenecks during validation and submission testing.

PROJECT

stc-server

+ Add environment

Production

All (2)Services (2)Env Groups (0)

Q Search resources in Production

SERVICE NAME	STATUS	RUNTIME	REGION	UPDATED	
STC-Server	✓ Deployed	Docker	Oregon	13min	...
stc-server	✓ Available	PostgreSQL 16	Oregon	1mo	...

+ New service

Fig 4.13: Backend deployment.

Chapter Five

Results, Recommendations and Conclusion

5.1 Results

The implementation of the Digital E-Invoicing and Tax Compliance Platform Prototype achieved the main objectives of the project. The system successfully demonstrated an end-to-end electronic invoicing workflow between the taxpayer client and the STC compliance server.

The taxpayer client was able to generate structured UBL 2.1 invoices, prepare invoice data, calculate invoice hashes, apply XAdES-style digital signatures, generate QR data, and submit invoices to the backend through clearance and reporting workflows.

The STC backend was able to receive submitted invoices, decode the Base64 invoice payload, validate the XML invoice structure, verify invoice hashes, verify digital signatures, check certificate binding, validate ICV and PIH chain continuity, store accepted invoices in PostgreSQL, and return suitable responses to the client.

The enrollment workflow also worked successfully. The taxpayer device generated the private key and CSR locally, submitted the CSR with a one-time enrollment token, and received an STC-signed device certificate. This confirmed that the prototype can establish trust between taxpayer devices and the STC server before invoice submission.

The clearance workflow returned a cleared invoice after successful validation, while the reporting workflow stored the invoice and returned an acknowledgment. The system also produced runtime logs that support auditing, debugging, and future monitoring.

Overall, the prototype proved that a digital tax compliance workflow can be implemented using a lightweight client-server architecture suitable for gradual adoption in Sudan.

5.1.1 Performance Results

Server performance was evaluated by sending repeated requests to the main backend endpoints: /health_check, /clear, and /report. The test measured request throughput, latency, failure rate, and response status codes.

PER REQUEST METRICS					
Name	# reqs	# fails	req/s	fail/s	
GET /health_check	698	0 (0%)	17.45	0.00	
POST /clear	6,764	0 (0%)	169.10	0.00	
POST /report	6,091	0 (0%)	152.27	0.00	
Aggregated	13,553	0 (0%)	338.83	0.00	
Name	Avg (ms)	Min	Max	Median	
GET /health_check	2.96	1	29	1	
POST /clear	63.28	8	264	60	
POST /report	49.87	5	238	47	
Aggregated	54.15	1	264	50	
Name	Status codes				
GET /health_check	698 [200]				
POST /clear	6,764 [200]				
POST /report	6,091 [202]				
Aggregated	7,462 [200], 6,091 [202]				

Fig 5.1: Server performance test output.

Table 5.1 server performance test output

Endpoint	Total Requests	Failures	Req/s	Avg (ms)	Status Codes
GET /health_check	698	0 (0%)	17.45	2.96	698 [200]
POST /clear	6,764	0 (0%)	169.10	63.28	6,764 [200]
POST /report	6,091	0 (0%)	152.27	49.87	6,091 [202]
Aggregated	13,553	0 (0%)	338.83	54.15	7,462 [200], 6,091 [202]

These results show that the Rust/Actix backend handled the prototype validation workflow with acceptable response time and no recorded failures.

The maximum invoice payload size observed during testing was approximately 11 KB, excluding other database fields. Based on this value, the estimated storage required for invoice payloads is shown in Table 5.2.

Table 5.2: Estimated invoice payload storage requirements.

Number of Invoices	Estimated Storage
1,000 invoices	11,000 KB (about 10.7 MB)
100,000 invoices	1,100,000 KB (about 1.05 GB)
1,000,000 invoices	11,000,000 KB (about 10.49 GB)

These values represent the invoice payload only. The real database size will be slightly larger because each invoice record also stores additional fields such as UUID, invoice hash, device identifier, invoice type, timestamp, database row overhead, indexes, logs, backups, and maintenance data.

The performance test was conducted in a local laptop-based environment using an Intel Core i5-6300U processor, 8 GB of RAM, and CachyOS Linux. The backend server, containerized PostgreSQL database, and load-testing tool were all executed on the same machine. Therefore, the measured results represent a local prototype test environment rather than a dedicated production server benchmark.

During the test, the backend memory usage remained approximately **25 MB**, which indicates that the Rust/Actix backend handled the tested workload with low memory consumption. However, because the database, backend, and load generator shared the same hardware resources, the results should be interpreted as prototype-level performance observations, not final production capacity measurements.

5.2 Recommendations

To improve the system and prepare it for future expansion, the following recommendations are proposed:

- Improve private key protection by using encrypted storage, operating system secure storage, smart cards, or hardware security modules instead of plain local PEM files.
- Add a full STC officer dashboard to allow authorized staff to view submitted invoices, taxpayer devices, validation results, and audit logs.
- Add stronger API authorization, role-based access control, rate limiting, and abuse protection before any production deployment.
- Expand the system to support more tax document types such as credit notes, debit notes, invoice cancellation, and invoice correction workflows.
- Perform larger-scale testing to evaluate system behavior under many taxpayers, concurrent invoice submissions, and higher transaction volume.
- Integrate the platform in the future with taxpayer registration systems, banking systems, customs systems, and business registry systems.
- Add advanced monitoring and observability tools for logs, metrics, alerts, and performance analysis.

5.3 Conclusion

This project presented a prototype for a Digital E-Invoicing and Tax Compliance Platform for the Sudan Taxation Chamber. The prototype demonstrated how businesses can generate compliant electronic invoices and submit them to an authority-side server for validation, clearance, reporting, storage, and auditing.

The system combined several important technologies and concepts, including UBL 2.1 XML invoices, XML canonicalization, SHA-256 hashing, XAdES-style digital signatures, certificate-based device enrollment, QR generation,

REST APIs, PostgreSQL persistence, and invoice chain validation using ICV and PIH.

The results show that the proposed system can improve invoice integrity, reduce opportunities for invoice manipulation, support better tax traceability, and provide STC with a foundation for digital tax compliance. Although the system is still a prototype and not a full national deployment, it successfully demonstrates the core technical workflow needed for a modern e-invoicing platform.

With further development, stronger security, larger-scale testing, official integration, and improved administrative tools, the prototype can be extended into a more complete electronic invoicing and tax compliance solution suitable for Sudan's future digital taxation infrastructure.

References

- [1] Reuters, “Khartoum’s shattered factories pick up the pieces as war rages,” Reuters, Oct. 2025. Available: <https://www.reuters.com/sustainability/sustainable-finance-reporting/khartoums-shattered-factories-pick-up-pieces-war-rages-2025-10-23> [Accessed 31/1/2026].
- [2] European Parliament and Council, “Directive 2014/55/EU of the European Parliament and of the Council on electronic invoicing in public procurement,” Official Journal of the European Union, 2014. Available: <https://eur-lex.europa.eu/legal-content/EN/TXT/?uri=CELEX:32014L0055> [Accessed 13/2/2026].
- [3] A. R. Hevner, S. T. March, J. Park, and S. Ram, “Design Science in Information Systems Research,” MIS Quarterly, vol. 28, no. 1, pp. 75–105, 2004.
- [4] K. Peffers, T. Tuunanen, M. A. Rothenberger, and S. Chatterjee, “A Design Science Research Methodology for Information Systems Research,” Journal of Management Information Systems, vol. 24, no. 3, pp. 45–77, 2007.
- [5] ETSI, XML Advanced Electronic Signatures (XAdES), ETSI TS 101 903, European Telecommunications Standards Institute. Available: https://uri.etsi.org/01903/v1.1.1/ts_101903v010101p.pdf [Accessed 20/2/2026].
- [6] World Bank, Digital Transformation of Tax and Customs Administrations, World Bank Group, Washington, DC. Available: <https://openknowledge.worldbank.org/entities/publication/8f69a6d3-4633-5dd4-9ddb-d1c21f315f23> [Accessed 10/3/2026].

- [7] OECD, Tax Administration 3.0: The Digital Transformation of Tax Administration, OECD Publishing, Paris, 2020.
- [8] Zakat, Tax and Customs Authority (ZATCA), Electronic Invoicing (FATOORA) Implementation and Technical Specifications, Riyadh, Saudi Arabia, 2021–2023. Available: https://zatca.gov.sa/en/E-Invoicing/Introduction/Guidelines/Documents/E-Invoicing_Detailed_Guideline.pdf [Accessed 10/5/2026].
- [9] United Nations Economic Commission for Europe, “Introducing UN/EDIFACT,” UNECE / UN/CEFACT Technical Documentation. Available: <https://unece.org/trade/uncefact/introducing-unedifact> [Accessed 10/4/2026].
- [10] X12, “X12 Transaction Sets and Supply Chain Transaction Flow,” X12 Standards Documentation. Available: <https://x12.org/flow/supply-chain> [Accessed 16/4/2026].
- [11] OASIS, Universal Business Language Version 2.1, OASIS Standard, 2013; ISO/IEC 19845:2015. Available: <https://docs.oasis-open.org/ubl/UBL-2.1.pdf> [Accessed 12/3/2026].
- [12] OpenPeppol, Peppol BIS Billing 3.0, OpenPeppol Documentation. Available: <https://docs.peppol.eu/poacc/billing/3.0/> [Accessed 20/4/2026].
- [13] European Commission, EN 16931 Electronic Invoicing Standard and Compliance Guidance, Brussels, Belgium. Available: <https://ec.europa.eu/digital-building-blocks/sites/spaces/DIGITAL/pages/467108950/EN+16931+compliance> [Accessed 15/3/2026].
- [14] ISO 20022 Registration Authority, “ISO 20022 Message Definitions,” ISO 20022 Financial Message Standard Documentation. Available:

- <https://www.iso20022.org/iso-20022-message-definitions> [Accessed 23/3/2026].
- [15] GS1, GS1 XML Technical and Functional User Guides, GS1 Standards. Available: <https://www.gs1.org/standards/gs1-xml/guideline/gs1-xml-technical-user-guide/3> [Accessed 20/4/2026].
- [16] OpenAPI Initiative, OpenAPI Specification Version 3.1.0, OpenAPI Initiative, 2021. Available: <https://spec.openapis.org/oas/v3.1.0.html> [Accessed 2/5/2026].
- [17] ETSI, Electronic Signatures and Infrastructures (ESI); XAdES Digital Signatures, ETSI EN 319 132-1, European Telecommunications Standards Institute. Available: https://www.etsi.org/deliver/etsi_en/319100_319199/31913201/01.03.01_60/en_31913201v010301p.pdf [Accessed 26/3/2026].
- [18] W3C, XML Signature Syntax and Processing Version 1.1, World Wide Web Consortium Recommendation. Available: <https://www.w3.org/TR/xmlsig-core1/> [Accessed 2/4/2026].
- [19] D. Merkel, “Docker: Lightweight Linux Containers for Consistent Development and Deployment,” Linux Journal, no. 239, 2014. Available: <https://www.linuxjournal.com/content/docker-lightweight-linux-containers-consistent-development-and-deployment> [Accessed 27/2/2026].
- [20] B. Burns, B. Grant, D. Oppenheimer, E. Brewer, and J. Wilkes, “Borg, Omega, and Kubernetes,” Communications of the ACM, vol. 59, no. 5, pp. 50–57, 2016. Available: <https://research.google/pubs/borg-omega-and-kubernetes/> [Accessed 7/3/2026].

- [21] Kenya Revenue Authority, Electronic Tax Invoice Management System (eTIMS) Guides and Taxpayer Information, Nairobi, Kenya. Available: <https://etims.kra.go.ke/> [Accessed 2/2/2026].
- [22] Federal Inland Revenue Service, TaxPro-Max Tax Administration Platform Information and User Guides, Abuja, Nigeria. Available: <https://taxpromax.firs.gov.ng/> [Accessed 12/3/2026].
- [23] A. S. Ismaiel Haj-Elamin, “STC Server Technical Documentation,” STC-Server GitHub repository, 2026. Available: https://github.com/ayman2ppp2/STC-Server/blob/master/TECHNICAL_DOCUMENTATION.md [Accessed 27/5/2026].
- [24] S. Y. Farah Balal, “STC Client Technical Documentation,” stc-client GitHub repository, 2026. Available: https://github.com/sameryousif/stc-client/blob/main/technical_doc.md [Accessed 27/5/2026].